We Gotta Combine!

Cost-Aware Routing of Open-Weight Specialist Models Across Software Engineering and Browser Tasks

Author name and university affiliation to be completed

25 October 2026 — pre-results manuscript

Contents

Manuscript status **4**

Abstract **5**

Acknowledgement **6**

1 Introduction **7**
1.1 Motivation . . . 7
1.2 The research problem . . . 7
1.3 Scope and terminology . . . 8
1.4 Research questions . . . 9
1.5 Intended contribution and the frontier objective . . . 9
1.6 Organization . . . 10

2 Foundations and related work **11**
2.1 Review scope and evidence quality . . . 11
2.2 From code generation to software engineering . . . 12
2.3 Selective prediction and learning to defer . . . 13
2.4 Calibration, discrimination and distribution shift . . . 14
2.5 Budget-aware routing and cascades . . . 15
2.6 Speculative decoding as inspiration, with different guarantees . . . 16
2.7 Distillation and progressive local capability . . . 17
2.8 Parameter-efficient adaptation and AI feedback . . . 18
2.9 Benchmark validity, contamination and harness effects . . . 19
2.10 Synthesis and research contribution . . . 20

3 System design and formalization **22**
3.1 Design principles . . . 22
3.2 Model registry, state and eligible actions . . . 22
3.3 Expected utility and customer price sensitivity . . . 24
3.4 Sequential delegation and state continuity . . . 25
3.5 Confidence-scheduled verification as an analogy . . . 26
3.6 Student learning mechanisms . . . 26
3.7 Adaptation and routing as separate interventions . . . 27
3.8 Cost model . . . 27
3.9 Evidence record . . . 28

4 Model and price landscape **29**
4.1 Candidate selection . . . 29
4.2 Price comparison and what it measures . . . 31
4.3 Published accuracy evidence: contextual, not a leaderboard . . . 32
4.4 Runtime readiness and the first bounded study . . . 33

5 Published open- and closed-weight software-engineering accuracy 35

6 Task-level prices and portfolio economics 37
6.1 The unit of purchase is a completed attempt . . . 37
6.2 Attributable historical cost per software-engineering task . . . 38
6.3 Illustrative prices for current catalog candidates . . . 38
6.4 Local and hosted infrastructure costs . . . 39
6.5 Adaptation, switching and the price-sensitivity policy . . . 40
6.6 Reproduction and evidence boundary . . . 41

7 Browser use and specialization across domains 42
7.1 What constitutes a separate domain . . . 42
7.2 Reusing browser research infrastructure . . . 43
7.3 Candidate models and limits of published evidence . . . 44
7.4 Held-out specialization and task features . . . 45
7.5 Measuring task value and cost . . . 46
7.6 Handoffs, adaptation and staged execution . . . 47

8 Customer adaptation, model placement and switching costs 49
8.1 The deployed expert is more than a model name . . . 49
8.2 What can be shared across customer adaptations . . . 49
8.3 Reuse established multi-adapter serving . . . 50
8.4 KV-cache continuity and the meaning of a switch . . . 51
8.5 A useful exception: activated LoRA on one base . . . 52
8.6 Select once, then switch at justified boundaries . . . 52
8.7 Desktop fine-tuning versus cloud adapters . . . 53
8.8 Measurements and adaptation lifecycle . . . 53

9 Cost-aware portfolios across task domains 55
9.1 Expanded research object and relation to the two-model study . . . 55
9.2 Related work and reusable infrastructure . . . 55
9.3 Candidate identity, objectives and price sensitivity . . . 56
9.4 Task specialization and portfolio size . . . 57
9.5 Whole-task routing and bounded handoffs . . . 58
9.6 Data acquisition, calibration and comparator selection . . . 58
9.7 Comparisons and confirmatory analysis . . . 59
9.8 What it means to approach the combined frontier . . . 60
9.9 Refreshing models and adapting customers . . . 60
9.10 Staged feasibility and execution boundary . . . 61

10 Software-engineering evaluation and the restricted two-model case 62
10.1 What “single LLM” means in this comparison . . . 62
10.2 Benchmark selection and target population . . . 63
10.3 Development data, test isolation and repository familiarity . . . 64
10.4 Common agent scaffold and execution controls . . . 65
10.5 Model eligibility and restricted-pair experimental arms . . . 65
10.6 Two-model router baseline and deployment policy . . . 67
10.7 Escalation and confidence features acquired during execution . . . 68
10.8 Outcomes and routing diagnostics . . . 68
10.9 Economic accounting and pricing comparisons . . . 69
10.10 Statistical analysis and sample-size planning . . . 70
10.11 Student adaptation and logit-distillation extensions . . . 71

10.12 Resource plan, execution stages and stopping rules 71
10.13 Reproducibility, artifacts and completion criteria 72

11 Analysis and reporting plan **74**
11.1 Estimands and success criteria 74
11.2 Accuracy and cost frontiers 75
11.3 Router quality as a prediction problem 75
11.4 Statistical uncertainty and dependence 76
11.5 Power and the role of the feasibility pilot 76
11.6 Prespecified outputs 77
11.7 Result-dependent discussion to be written later 77
11.8 Portfolio and switching diagnostics 78

12 Reproducibility, ethics and limitations **79**
12.1 Implementation state 79
12.2 Reproducibility package 80
12.3 Task management and publication 80
12.4 Data and research ethics 81
12.5 Construct validity 81
12.6 Internal validity 82
12.7 External validity 82
12.8 Resource and completion boundaries 83

Results and discussion reserved for completion **84**

Appendix: empty reporting templates **85**
Main system comparison 85
Routing and student diagnostics 85
Run manifest checklist 86

References **87**

Manuscript status

This document is a master's thesis manuscript in artificial intelligence, prepared before the proposed software-engineering and browser-task evaluations. Its research questions, background, system design, experimental protocol, analysis plan and limitations are written in advance. **The empirical results, result-dependent discussion and empirical conclusion are intentionally not written.** No claim that the proposed model portfolio improves accuracy, reduces measured cost or benefits from adaptation is made. The revised scope replaces a fixed local/remote pair with a pool of eligible specialist models; the earlier two-model protocol remains a restricted baseline and integration pilot.

A historical programming-problem replay and software-validation artifacts exist in the accompanying repository. They are development material for a different task distribution, and their measurements are not presented as evidence for this thesis's hypotheses. Each prospective domain study requires its own frozen model catalog, evaluation records and analysis. Published third-party benchmark accuracy and task-cost observations are background evidence. Illustrative token-cost calculations are explicitly labelled scenarios, not measured task prices or experiment results.

The source and pricing review has a cutoff of **25 October 2026**. A model catalog is a dated snapshot, not a promise that every listed model is deployable on the available hardware or that a floating API alias will remain unchanged. Secondary price observations require vendor confirmation before purchasing inference. The existing US$100 Runpod authorization funds a bounded feasibility pilot; it does not fund the entire proposed confirmatory matrix or additional closed-provider API spending.

Author, institution, degree wording, supervisor, institutional declarations and submission date are to be completed by the author. The manuscript does not invent those details or represent that an institution has approved the research.

Abstract

Language models differ in their strengths, deployment requirements and cost. A model suited to backend repair need not be the best choice for frontend development or browser interaction; parameter count and a single aggregate benchmark do not establish a universal ordering. An assistant can instead select from a portfolio of open-weight specialists, with optional closed-model references, according to expected task performance, total cost, latency and customer constraints. Small models may run locally or remotely, and customer-specific adaptation may improve selected members of the portfolio.

This thesis proposes a controlled evaluation of cost-aware portfolio routing across software-engineering and browser tasks. The primary software-engineering comparison tests a frozen multi-model policy against a strong fixed model selected on development data under a common agent harness. Domain-specific selection, two-model routing, random allocation and simple rules provide additional controls; browser interaction supplies a separate transfer study. Accuracy–cost frontiers are reported in dollars per assigned task and per successful task, with workload-specific success definitions and complete accounting for failures. Task-level model selection is the default. Phase-level and per-call switching are evaluated separately, including model loading, context reprocessing and state-transfer costs.

The design separates portfolio selection, router recalibration and model adaptation. New releases are admitted through versioned development evaluations, while final tests remain unavailable for selection. Customer adapters are associated with compatible base-model revisions; shared training data does not imply interchangeable adapter weights or key–value caches. The study measures whether additional specialists or adaptation improve the attainable frontier enough to justify their operational cost. Published model and price evidence informs candidate selection but does not establish a universal best combination. The intended contribution is a reproducible account of how closely a deployable policy can approach the capabilities of its eligible model pool at a chosen price sensitivity. No empirical outcomes for that hypothesis are reported.

Keywords: model portfolios; software-engineering agents; browser agents; cost-sensitive routing; calibrated confidence; customer adaptation; LoRA; open-weight language models.

Acknowledgement

The project’s guiding sentiment is: “Believe in me who believes in you.”

Chapter 1

Introduction

1.1 Motivation

An assistant's useful output is completed work: a repository change that passes its acceptance checks, a browser workflow that reaches the requested state, or a frontend that meets a specified interaction requirement. Generating a plausible answer is only part of that process. The model operates through tools, observations, execution environments and stopping rules, so its performance must be measured as part of an agent system. A single language-model score does not fully characterize the service the customer receives.

Model capabilities are heterogeneous. Backend debugging, visual frontend work, unfamiliar programming languages, long repository contexts and browser navigation can require different strengths. Some tasks need visual grounding; others can be completed efficiently from structured text. A smaller specialist may outperform a larger general model on one workload while failing on another. These possibilities motivate empirical comparison, not an assumption that specialization or parameter count guarantees superior performance.

The proposed TTGL assistant therefore selects among a portfolio of eligible models. The portfolio may include several small models, larger general models, browser specialists and compatible customer-adapted variants. A model can run on a desktop, a shared GPU service or a managed endpoint. Model size, weight availability, execution location and customer billing are separate attributes. An open-weight model served remotely still incurs infrastructure cost; local execution consumes memory, energy and time even when the customer pays no per-call fee.

The working title, **We Gotta Combine!**, expresses the hypothesis that complementary models can be more useful together than a fixed choice. Here, combination primarily means selecting an appropriate model for each task. It does not require running every model, voting on every answer, merging their weights or switching after every generated response. Those interventions have different costs and require separate experiments.

1.2 The research problem

The central problem is to predict which eligible model offers the best expected outcome for a task at the customer's price sensitivity. An inexpensive model may

be sufficient; a more expensive model may provide enough additional success probability to justify its cost; or neither may have a reasonable chance of success within the budget. The router must learn those conditional differences. Detecting that a task is difficult is insufficient if all candidates fail on it.

A task's full price is not its advertised output-token rate. Repeated context processing, tool calls, failed attempts, verification, model loading, cache behavior, queueing and infrastructure allocation all affect the final bill or economic cost. The study therefore reports both dollars per assigned task and dollars per successful task. The latter includes the cost of failures in its numerator. Price scenarios derived from token rates are useful for planning, but measured task costs require actual traces and an explicit accounting boundary.

Choosing the portfolio itself is part of the problem. Adding a specialist may improve coverage while increasing data acquisition, model residency, adapter training and maintenance costs. Several small remote models may be practical for a shared service but unsuitable for a memory-constrained laptop. Conversely, one well-adapted local model may be preferable when offline operation or privacy dominates small average accuracy gains. The experiment treats the number and sizes of models as choices to evaluate, not fixed consequences of the words "small" and "large."

Model releases and workloads also change over time. A newly released model cannot be assigned a reliable router score merely because its provider reports a better benchmark. It must pass interface and licensing checks, acquire development measurements and enter a versioned pool. Updating router parameters can then be much cheaper than retraining every generative model, but it still requires evidence. Customer adaptation and portfolio refresh are separate interventions whose effects must be identified independently.

1.3 Scope and terminology

A **model configuration** includes the base checkpoint or service revision, tokenizer and prompt template, precision, inference settings, adapter where applicable, runtime and endpoint. Two deployments of the same checkpoint can have different cost and latency; an adapted checkpoint can have different accuracy. A **portfolio** is the finite, frozen set of configurations eligible within a study epoch. A **router** predicts their conditional value and chooses an eligible action subject to budget, permissions, modality and device constraints.

A **single-model agent** may invoke many tools and make many model calls, but keeps the same configuration throughout its trajectory. A **portfolio-routed agent** can choose a different configuration for different tasks. A **switching agent** can also transfer an ongoing task between configurations. The initial portfolio experiment uses task-level selection so that routing quality can be assessed before adding the confounding effects of state transfer. Two-model local/remote routing is a restricted case, retained as a useful baseline and inexpensive integration pilot.

"Open-weight" means weights are available under a published license; it does not establish all properties of an open-source AI system or grant unrestricted commercial use. Weight licenses, source-code licenses, API terms and training permissions are recorded separately. "Free model" is consequently not used as a synonym for zero total cost. An open-only pool and a pool permitting closed APIs

can be compared under clearly declared availability rules.

The primary confirmatory outcome is repository-level issue resolution under a pinned software-engineering evaluator. Browser interaction is a separate domain study using its own common scaffold and task-success definition. Frontend development is not interchangeable with browser navigation: producing an application and operating an existing website require different acceptance checks. Programming language, frontend/backend role and task type are prespecified, potentially overlapping strata. Domain-level scores are not pooled into one accuracy number unless the workload weights and normalization have been declared in advance.

1.4 Research questions

The thesis organizes the empirical work around six questions:

1. **Portfolio value:** Can a frozen multi-model policy retain success within a prespecified tolerance of a strong fixed model selected on development data while reducing total cost per task?
2. **Routing quality:** Does learned allocation outperform random, domain-based, simple-rule and two-model allocation at comparable expected budgets, and are predicted outcomes and costs calibrated?
3. **Complementarity:** How do the attainable frontier and the selected model mix vary with frontend/backend work, programming language, browser interaction, portfolio size and price sensitivity?
4. **Switching granularity:** When do task-level selection, phase-level handoff or per-call switching provide the best tradeoff after accounting for context replay, cache validity, model loading and handoff failures?
5. **Customer adaptation:** Which small models, if any, justify per-customer adapters or full-model adaptation, and how much improvement comes from changed model capability versus router recalibration?
6. **Continual selection:** Can a versioned admission and recalibration process incorporate new model releases without losing calibration, leaking test data or consuming more resources than the resulting savings justify?

The ensemble methodology defines one confirmatory comparison and a restricted statistical claim. Browser transfer, wider portfolio sweeps, switching and customer adaptation are prespecified extensions unless separately funded and registered. This separation keeps the central thesis answerable while retaining the broader product direction.

1.5 Intended contribution and the frontier objective

The thesis does not claim to invent model routing, calibration, parameter-efficient adaptation or agent harnesses. Its intended contribution is a controlled study of their interaction across task domains, with a reproducible per-task economic account and explicit operational costs. Existing libraries supply the agent loops, browser interaction, model transport, inference and training. New code is limited to the research-specific routing, integration and evidence record where maintained tools do not provide them.

The combined capability of a model pool is an attractive upper bound, not a

promised deployed frontier. An outcome-informed oracle can choose a successful model after seeing every candidate's outcome; a real router must choose before observing that outcome. Correlated failures, prediction error, insufficient training data and the cost of acquiring routing features can leave a substantial gap. Even the oracle only characterizes the finite evaluated pool and task distribution, not every available open model or every future release.

A useful result could therefore be positive, negative or qualified. Extra models might improve one language but provide no benefit elsewhere. A domain lookup might match an expensive learned router. State handoff might erase the gains from finer-grained selection. A customer adapter might improve its target workload while damaging other tasks or never amortizing its training cost. The protocol is designed to expose these possibilities rather than select only favorable demonstrations.

The repository already contains a historical coding-problem replay, data validation, teacher-example preparation and a training/inference scaffold. These do not establish the performance of the proposed portfolio, browser agent or latest checkpoints. New integration and empirical execution remain necessary. The approved single-80-GB-GPU, US$100 Runpod allocation supports bounded feasibility work; it does not imply that the expanded confirmatory study or additional paid APIs have been funded.

1.6 Organization

The foundations chapter reviews routing, calibration, speculative inference, distillation and agent evaluation. System design defines the model portfolio and decision interface. The dated model landscape and historical accuracy evidence motivate candidate selection, while the task-economics chapter separates published task costs from hypothetical price scenarios. Browser evaluation and customer adaptation receive dedicated chapters. The ensemble protocol controls the expanded experiment, and the software-engineering chapter retains detailed benchmark procedures and the restricted two-model pilot. The analysis plan and reproducibility chapter specify reporting, limitations and completion criteria. Empirical results and result-dependent conclusions are reserved for later completion.

Chapter 2

Foundations and related work

2.1 Review scope and evidence quality

This chapter develops the conceptual basis for a portfolio-routed assistant spanning software engineering and browser interaction and identifies the claims that the proposed experiment can test. Its scope is deliberately broader than model routing alone. A useful system must choose a model, obtain a usable solution through an interaction interface, verify that solution, and account for the resources consumed by both successful and unsuccessful attempts. The relevant literature therefore spans selective prediction, learning to defer, LLM routing, agent-computer interfaces, code evaluation, knowledge distillation and parameter-efficient adaptation.

The review is a focused narrative synthesis, not a systematic review or meta-analysis. Source discovery combined exact-title lookup, author-maintained repositories and backward citation following. The review cutoff and retrieval date are 25 October 2026. Exact-title lookup targets included “RouteLLM: Learning to Route LLMs with Preference Data”, “FrugalGPT”, “Consistent Estimators for Learning to Defer to an Expert”, “On Calibration of Modern Neural Networks”, “Distilling the Knowledge in a Neural Network”, “Dynamic Speculation Lookahead”, “SWE-agent” and “SWE-bench”. The bibliography records the corresponding titles, stable paper identifiers and software revisions, allowing another reviewer to reconstruct the selected corpus. No broad scholarly-index search or database hit-count screening was conducted.

Inclusion required a source to define a relevant method, benchmark or implementation, with identifiable authorship and primary provenance. Foundational papers were retained even when their experimental domain preceded LLMs; contemporary papers and software were included when they changed the available routing, training or evaluation choices. Exclusions covered untraceable claims, advertising or leaderboard rank without evaluation context, duplicate descriptions of the same artifact, and material unrelated to the study’s decision or validity questions. These rules bound a purposive review rather than guarantee exhaustive coverage. The research gap is consequently a motivated empirical question, not a claim that no related system has ever been studied.

Evidence quality varies across sources. Relevant definitions, methods and discussion were inspected in primary full texts for sixteen works: calibration, SelectiveNet, learning to defer, FrugalGPT, RouteLLM, speculative decoding,

dynamic speculation, distillation, LoRA, QLoRA, DPO, Constitutional AI, DeepSeek-R1, SWE-agent, SWE-bench and SWE-bench Pro. This is targeted section-level reading, not a claim that every proof, appendix and empirical table was independently audited. DeepInfer's confidence head and scheduling code were inspected at a pinned revision, and its claims remain implementation evidence. Other benchmark and SDK references rely on official repository documentation; their bibliographic notes identify this narrower evidence. The review does not treat authors' reported improvements as a replication or as predicted thesis outcomes.

The expanded portfolio and adaptation chapters add targeted primary-text reviews of RouterBench, GraphRouter, BEST-Route, LLMRouter, S-LoRA, Punica and activated LoRA. The browser chapter reviews official benchmark, model and integration documentation with explicit source-scope notes. Their separate registers preserve inspected versions and hashes; a documentation citation is not represented as a full-paper review.

Fast-moving software requires a second form of provenance. A paper's method, a repository's current code and a hosted product's current behavior need not coincide. This chapter separates paper citations from versioned software-documentation citations. SWE-bench, mini-SWE-agent and SWE-bench Pro documentation were inspected on 25 October 2026; their relevant revisions are recorded in the bibliography. Descriptions below establish the documented scope of these systems, not independent performance measurements. This pre-results manuscript makes no claim about the proposed system's measured accuracy, cost or training benefit.

2.2 From code generation to software engineering

A code-generation task usually supplies a specification and asks for a function or program that satisfies a collection of tests. HumanEval made executable functional correctness a practical evaluation target, while subsequent work such as EvalPlus strengthened the tests used to assess generated functions (M. Chen et al. 2021; Liu et al. 2023). These settings isolate an important capability, but successful synthesis of a self-contained function is only one component of software development.

A repository-level software-engineering task supplies an issue and a codebase at a particular revision. The system must identify relevant files, understand interfaces, modify implementation code, preserve existing behavior and produce a patch that survives evaluation. SWE-bench operationalizes this setting through real repository issues and corresponding evaluation machinery (Jimenez et al. 2024). BigCodeBench occupies a complementary position by emphasizing diverse library calls and complex instructions within function-level problems (Zhuo et al. 2024). LiveCodeBench provides temporally organized programming tasks useful for contamination-aware coding evaluation (Jain et al. 2024). None of these benchmark categories should be substituted for another without narrowing the claim accordingly.

An LLM and an agent are also distinct experimental objects. An LLM maps context to a distribution over outputs. An agent couples that model to tools, state, stopping rules, retrieval, execution and feedback. A model's repository-task performance is therefore conditional on an agent-computer interface and its resource limits.

SWE-agent directly studies that interface, and its current maintainers recommend the simpler mini-SWE-agent as the successor for ongoing use (Yang et al. 2024; SWE-agent Team 2026). OpenHands supplies a further example of separating agents, tools, conversations, workspaces and server interfaces within a reusable SDK (Wang et al. 2025). These systems support the methodological choice to reuse a maintained harness rather than invent a new one merely to demonstrate routing.

The appropriate primary comparison is thus a dual-model agent against single-model agents using the same harness, repository snapshot, available tools and evaluation rules. Comparing a tool-using dual system against a single bare prompt would confound routing with access to feedback and execution. A no-tools LLM can remain an informative secondary ablation, provided it is labelled as a different interface condition. Likewise, a benchmark score obtained with one vendor's proprietary agent should not be treated as the score of its underlying model under the thesis harness.

2.3 Selective prediction and learning to defer

Selective prediction studies a predictor that may abstain rather than produce an answer for every input. Its central quantities are coverage, the fraction of inputs accepted, and selective risk, the error rate among accepted inputs. SelectiveNet integrates prediction and rejection within a learned system (Geifman and El-Yaniv 2019). This perspective helps distinguish a useful local model from a merely overconfident one: increasing local coverage is desirable only if the accepted local work maintains adequate quality.

An assistant with a remote model is more accurately viewed as a learning-to-defer system because abstention invokes another decision-maker. Learning-to-defer formulations explicitly model the expert alternative rather than assume that rejection is costless (Mozannar and Sontag 2020). Here the remote model is an imperfect expert. It may solve a problem that defeats the local model, fail alongside it, or occasionally damage a solution the local model would have produced correctly. The experimental target must represent all of these cases.

Let X denote information available when a routing decision is made, and let $Y_s, Y_l \in \{0, 1\}$ denote acceptance of a solution generated by the small and large systems under specified protocols. The local-success probability is $p_s(x) = P(Y_s = 1 \mid X = x)$. A policy that delegates whenever $p_s(x)$ is low assumes that local failure predicts useful remote intervention. A more directly aligned quantity is

$$g(x) = \mathbb{E}[Y_l - Y_s \mid X = x].$$

This expected benefit can be positive, zero or negative. It differs from local uncertainty because a difficult problem can defeat both systems. When cost varies across tasks, delegation can instead depend on expected benefit relative to incremental cost, with explicit constraints on latency, budget and permitted destinations. This formulation is a decision rule to be evaluated; it does not establish that a cheaply estimated $g(x)$ will generalize.

Paired model outcomes on the same task provide useful training information, but their interpretation depends on the execution design. If models receive

different tools, feedback or stopping budgets, the outcome difference describes different systems rather than model substitution alone. Repeated stochastic samples provide an estimate of each system's task success distribution, not interchangeable independent tasks. Model sampling seeds also need not pair meaningfully across different providers. The experiment should preserve task-level pairing while recording the conditions that generated each outcome.

Operational logging introduces selective labels: a deployed router observes the remote outcome mainly on tasks it already chose to escalate. Training solely on those logs can reinforce its original selection bias. Controlled paired data, offline shadow evaluations or carefully budgeted exploration can provide information about counterfactual choices. Such exploration must be distinguished from extra user-visible attempts, and its cost belongs in the system's accounting.

2.4 Calibration, discrimination and distribution shift

Confidence is useful only after its target has been defined. Next-token probability expresses the model's distribution over a token; it is not automatically the probability that a completed patch resolves a repository issue. Natural-language statements of confidence introduce additional sensitivity to wording and prompting. A learned correctness predictor must instead be evaluated against the outcome it purports to estimate, such as benchmark resolution, successful local completion or beneficial remote delegation.

Calibration describes agreement between probabilities and observed frequencies. Guo et al. examine calibration in modern neural networks and motivate held-out calibration procedures such as temperature scaling (Guo et al. 2017). The general lesson applies to routing even though the original classification setting differs from software engineering: a score that orders examples well need not provide reliable probabilities, and a base-rate predictor can be reasonably calibrated while offering little discrimination. Ranking, calibration and final decision quality therefore require separate measurements.

For correctness predictors, reliability diagrams, Brier scores and carefully specified calibration-error summaries are useful complements to discrimination measures. Expected calibration error depends on binning and sample size; a single number can conceal poor behavior in a rare but consequential task stratum. AUROC evaluates a ranking over binary outcomes, while precision-recall can be more informative under severe class imbalance. Neither substitutes for the system-level question of how much accepted work is obtained for a given resource budget. Risk-coverage, resolution-cost and regret curves connect prediction quality to actual routing behavior.

Calibration and decision thresholds must be fitted on data disjoint from final evaluation. Selecting a threshold on the test set to achieve an attractive cost or quality point is part of training, even if model parameters remain fixed. A fixed threshold chosen on calibration data should be evaluated with its realized test exposure; an exact-budget allocation over a scored batch should be described as batch selection. These are different deployment settings.

The local/remote setting adds several sources of distribution shift. New repositories may use different languages, dependencies and issue conventions. Model updates change capability and output style. Quantization, context

truncation, retrieval and tool limits can alter success even when the checkpoint name is unchanged. Distilling the student changes the relationship the router is intended to predict. Consequently, a later improvement in the composite system must be decomposed into student changes and router recalibration. A frozen-router comparison and a separately recalibrated-router comparison answer different causal questions.

2.5 Budget-aware routing and cascades

FrugalGPT formulates a budget-constrained cascade whose scorer evaluates a query and generated answer; it learns a sequence of APIs and acceptance thresholds. RouteLLM instead predicts preference for the stronger model before generation, using pairwise preference data and a tunable selection threshold (Lingjiao Chen, Zaharia, and Zou 2024; Ong et al. 2025). Its full paper distinguishes reward scoring after generation from routing before generation and evaluates performance recovered relative to the strong/weak gap. These works establish useful baselines and software components. They do not establish that conversational preferences transfer directly to executable repository correctness. The label, task distribution, candidate model pair and feature budget all matter.

The distinction also limits which published metrics transfer unchanged. RouteLLM uses strong-model call share as a cost proxy and normalizes performance by the gap between its candidate models. In an agent loop, individual calls have different token counts and one task can require many calls; a small or negative baseline performance gap also makes normalized recovery difficult to interpret. The thesis therefore retains direct resolution and monetary-cost measurements, while using call share and normalized summaries only with their assumptions stated. FrugalGPT's answer-dependent scorer provides a closer conceptual baseline for verification-triggered escalation, but its query-answering experiments do not establish the cost of repository execution and repair loops.

A prompt-only router acts before generation and pays for one selected model. A cascade first runs a cheaper model, observes its output or execution feedback, and then decides whether to invoke a stronger model. The cascade obtains richer evidence but incurs the cost of the initial attempt even when it escalates. It can also lengthen the remote request by including failed edits, tool traces and summaries. Describing both designs as the same remote-call fraction would hide a meaningful cost difference.

The relevant quantity is total cost per task and per accepted task. For API models, input, cached input, output and provider-specific reasoning charges can differ. For self-hosted models, GPU allocation, idle time, memory requirements, batching and operational overhead determine cost. The local device also consumes electricity and user time even when it generates no cloud invoice. Training, teacher-data generation and artifact distribution add fixed or amortized costs that can dominate at small deployment scale. These terms motivate a price ledger and measured token accounting rather than translating a call-count reduction into an asserted monetary saving.

Fair model comparison requires more than a list of prices. Two systems at the same monetary budget can process different token volumes or make different numbers of tool calls. A model with a low output-token price may generate longer answers or require more repair attempts. Cache behavior depends on request

patterns and provider semantics, and idle self-hosted capacity can reverse an apparent advantage measured at full utilization. The proposed study should therefore report both the observed resolution-cost frontier and sensitivity to clearly labelled deployment assumptions.

An oracle router that selects the better observed outcome for each task is useful for measuring remaining opportunity. It is not a deployable competitor and is especially optimistic when selection uses noisy estimates from a finite number of samples. Random routing at matched exposure or matched expected cost remains essential: it reveals whether the learned allocation adds value beyond simply mixing two systems. A simple interpretable rule provides another baseline against which extra routing complexity must justify itself.

2.6 Speculative decoding as inspiration, with different guarantees

Speculative decoding uses a cheaper draft process to propose tokens that a target model verifies (Leviathan, Kalman, and Matias 2023). Dynamic speculation changes the amount of proposed work instead of using a fixed draft length. The DISCO paper specifies a learned classifier using top draft probabilities, entropy and token position to decide when drafting should stop (Mamou et al. 2024). The related Hugging Face/Intel implementation article also describes a draft-confidence stopping rule. These are related mechanisms with different predictors, not one interchangeable confidence algorithm. Both allocate computation according to the chance that cheap speculative work will be useful.

DeepInfer supplies a more recent implementation of this idea. Its documented DSpark path includes a learned confidence head for draft-token acceptance, cumulative estimates for surviving prefixes, runtime cost curves and confidence-scheduled verification. The scheduler allocates a verification budget across requests and records acceptance and confidence-error statistics (DeepSeek-AI 2026). The inspected source also imposes dependencies on which historical estimates may influence scheduling. These details make the method more specific than a generic instruction to trust a small model when its confidence is high.

The analogy to task routing is productive at the level of resource allocation: estimate useful work, compare expected benefit with cost, and check predictions against outcomes. The acceptance event and its guarantees are different. Token verification can be part of a sampling procedure designed to preserve the target distribution. Accepting an entire locally generated patch after incomplete tests does not preserve the distribution of patches the remote model would have generated, and it does not guarantee semantic correctness. A remote reviewer is itself fallible.

Dependence is also different. A speculative token prefix has a particular sequential structure, whereas software subtasks may share files, assumptions and hidden failure causes. Multiplying marginal confidence scores for several edits does not generally produce a valid probability that their combined patch succeeds. A router may instead predict final task utility directly or use explicitly conditional transition estimates. Either approach requires empirical calibration on the actual decision granularity.

The thesis therefore treats confidence-aware speculative scheduling as a design

inspiration, not as a theorem guaranteeing accurate task routing. Token-level speculative acceleration and task-level model selection should be evaluated as separate interventions if both are implemented. The former concerns inference efficiency under a particular serving engine and hardware configuration; the latter concerns which system should perform software-engineering work. Their gains cannot be added without measuring their interaction and overhead.

2.7 Distillation and progressive local capability

Knowledge distillation transfers behavior from a teacher to a student (Hinton, Vinyals, and Dean 2015). In this application the desired effect is increased local task success, enabling the routing policy to rely less often on remote inference at a fixed quality requirement. A reduction in remote usage alone is insufficient evidence of distillation: it could be caused by changing a threshold or accepting more local failures. Student improvement must be established independently on untouched tasks before evaluating the composite system.

Sequence distillation and token-distribution distillation impose different data requirements. Sequence distillation trains on teacher-generated responses, patches or observable tool trajectories. It can proceed without access to teacher logits and is compatible with different teacher/student tokenizers, provided the examples are represented correctly for the student. Token-level distillation typically compares teacher and student predictive distributions, often using a temperature-scaled divergence combined with supervised loss. That comparison requires a meaningful correspondence between token events. A DeepSeek tokenizer and a Qwen tokenizer cannot be assumed aligned merely because both process the same source text. Sparse top-token API outputs also do not automatically supply a full-vocabulary teacher distribution.

Verification changes the training distribution. Selecting teacher outputs that pass tests can improve the reliability of demonstrations, but it favors tasks for which the teacher and available tests succeed. It may omit difficult requests and retain solutions that exploit weaknesses in the test suite. A verification pipeline should therefore preserve selection counts and reasons, separate training verifiers from final evaluation, and maintain an explicit account of the examples that were never eligible. Teacher-generated code should not be treated as correct solely because it came from the larger model.

The inspected DeepSeek-R1 paper describes rule-based accuracy rewards, rejection-filtered training data and SFT-based distillation of smaller models. Its discussion provides a concrete reason to include supervised distillation as a strong baseline before adding more complex reinforcement-learning stages (DeepSeek-AI 2025). Those observations arise from the authors' particular models, data and compute regime. They do not establish that distillation always dominates direct reinforcement learning, that a small customer dataset is sufficient, or that improvement on reasoning questions implies improved repository repair.

The software-engineering setting raises a further transfer issue. A final patch may omit the process needed to discover it: repository search, diagnosis, testing and revision. If the student is intended to run an agent loop, final-answer imitation and trajectory imitation should be distinguished. Observable tool actions and concise task-state summaries can be legitimate training targets; the design does not require access to a provider's private reasoning trace. Repository instructions

and retrieval remain lower-cost personalization baselines that should be evaluated before attributing benefits to model training.

2.8 Parameter-efficient adaptation and AI feedback

LoRA represents weight updates with low-rank trainable components while retaining a frozen base model (E. J. Hu et al. 2021). QLoRA combines a quantized frozen base with parameter-efficient adaptation to reduce training memory (Dettmers et al. 2023). These techniques address how parameters are updated; they do not specify which demonstrations are trustworthy or whether the student improves. They also do not remove the memory and compute required for forward passes, optimizer state, activations and evaluation.

The deployment artifact is part of the treatment. An adapter trained in one framework may need merging or conversion before it can be used by the selected local runtime. Quantization can alter outputs, tool formatting and task success. A fair evaluation must include the actual artifact intended for the user's hardware rather than report only unquantized server performance. Base checkpoint, tokenizer, chat template, adapter revision and export procedure should therefore be recorded together.

Direct Preference Optimization learns from preferred and rejected responses relative to a reference policy, providing an alternative to explicitly fitting and optimizing a separate reward model (Rafailov et al. 2023). Reinforcement learning from AI feedback identifies the source of feedback, not one particular algorithm. AI-generated rankings, executable-test preferences and human review are distinct label sources even when supplied to the same optimizer. Constitutional AI combines supervised critique/revision with preference-model training and reinforcement learning; its reported setup retains human helpfulness labels while using AI feedback for harmlessness (Bai et al. 2022). This distinction prevents the mistaken inference that the original method removes every human supervisory input. DeepSeekMath and DeepSeek-R1 motivate verifiable objectives for reasoning tasks (Shao et al. 2024; DeepSeek-AI 2025). None of these sources establishes that an AI judge reliably substitutes for independent code execution.

For this thesis, the controlled sequence is a frozen student, verified-teacher SFT, and only then an optional preference-learning extension with a specified label source. Equalizing base model, data splits, example/token budgets and evaluation conditions helps distinguish the benefit of better supervision from additional compute. If online reinforcement learning is added, candidate-generation and verification costs must be included. Reward hacking, incomplete tests, correlated teacher/judge errors and catastrophic forgetting remain threats requiring separate measurements.

Personal or customer-specific adapters add fragmentation: small datasets may not support robust calibration, and each base/adapter combination needs deployment and rollback management. Training consent, teacher-output rights and artifact licenses are separate from owning the application distribution channel. These considerations motivate an opt-in, versioned data pipeline and a comparison with simpler retrieval/instruction personalization. They are implementation conditions for the study, not evidence that every customer requires an individual adapter.

2.9 Benchmark validity, contamination and harness effects

Executable evaluation is stronger than relying solely on an LLM judge, but its interpretation remains bounded by the tests. EvalPlus demonstrates why test coverage and benchmark revisions matter for code-generation assessment (Liu et al. 2023). The same principle applies to repository repair: a passing patch may satisfy the selected tests while introducing untested regressions, and a failing run can reflect an environment defect rather than reasoning failure. Infrastructure, extraction, tool-protocol and solution failures should be recorded separately without using post-hoc exclusions to improve a model's score.

SWE-bench Verified is documented as a subset of 500 problems that software engineers confirmed were solvable (SWE-bench Team 2024). That validation improves the evaluation target but does not make every current model comparison contamination-free or every hidden test complete. The inspected SWE-bench documentation also warns that evaluation results are cached by run identifier and instance identifier, rather than changed patch content. Reusing identifiers across treatments can therefore silently reuse an earlier result. Unique run identities, patch hashes and current-run artifact checks are necessary to establish that the intended experiment actually executed.

SWE-bench Pro extends the repository-task setting toward longer-horizon work (Deng et al. 2025). Its currently inspected V2 documentation specifies a changed public set of 642 tasks across 11 repositories, a locked agent phase and regrading of the captured patch in a fresh sandbox (Scale AI 2026). The documented changes from V1 include task instructions, tests, reference patches and images. Consequently, scores from those releases cannot be merged as though they were measurements on one unchanged benchmark. Reference solutions and verifier fixtures must also remain outside the agent's accessible task context.

Public benchmark age and model release date do not identify a model's training exposure with certainty. Time-windowed collections such as LiveCodeBench improve experimental control, but their contamination properties depend on the model checkpoint and the task's prior availability (Jain et al. 2024). Repository-level splits can limit direct transfer of conventions and solutions between router training and testing; temporal splits answer a different generalization question. Both require deduplication and documented grouping rules. Foundation-model pretraining contamination may remain unobservable and should be reported as a residual limitation rather than dismissed by a dataset title.

Finally, published leaderboard accuracy is a system-level observation. Model revision, scaffold, retrieval, tool budgets, test split, number of attempts and reasoning settings can all differ. A pricing table can summarize current offerings, but plotting an incomparable headline score against an unrelated API price does not establish an accuracy-cost frontier. The thesis should distinguish contextual published claims from controlled same-harness measurements and should avoid ranking systems whose evaluation conditions cannot be aligned.

2.10 Synthesis and research contribution

The literature already supplies routing frameworks, deferral objectives, calibrated predictors, agent harnesses, executable benchmarks and efficient adaptation methods. The defensible contribution is their controlled evaluation in a particular setting: whether a portfolio of specialist configurations can complete tasks at lower total cost while meeting an explicit quality requirement, and whether a router can allocate that work better than fixed, domain-based and restricted two-model alternatives.

Three interactions deserve direct empirical treatment. First, confidence and task difficulty need not identify remote benefit; many candidate models can fail on the same difficult tasks. Second, richer feedback can improve decisions while eliminating savings through local attempts, verification or duplicated context. Third, student adaptation changes both capability and calibration, so apparent improvement in the composite system must be separated into model and routing effects. The experiment's ablations, accounting and statistical plan should correspond to those interactions rather than merely compare the final product with an unspecified LLM baseline.

Literature strand	What it supplies	What the thesis must establish
Selective prediction and deferral	Coverage/risk concepts and an imperfect-expert formulation	Local acceptance and useful delegation on repository tasks
FrugalGPT and RouteLLM	Routing/cascade methods and reusable baselines	Transfer from existing labels and tasks to executable software-engineering outcomes
Multi-model routing and shared-adapter serving	Candidate-pool methods and maintained deployment tools	Complementarity, model admission and per-customer lifecycle economics
Browser and visual benchmarks	Reusable interaction contracts and task evaluators	Domain-specific selection without conflating navigation and frontend coding
Calibration research	Methods to relate scores to frequencies	Reliability under repository shift and model adaptation
Speculative inference	Confidence-guided allocation of verification compute	A valid task-level policy without importing token-level guarantees
Distillation and LoRA	Transfer objectives and efficient parameter updates	Held-out student improvement after deployment conversion
Preference and verifiable-feedback learning	Alternative supervision and optimization mechanisms	Gains beyond SFT after counting extra data and compute
SWE benchmarks and agent interfaces	Reproducible tasks, tools and evaluation protocols	Comparable model/hybrid results under one frozen interface

This contribution is conditional and falsifiable. A router that does not outperform a matched-cost baseline, a portfolio policy that violates the quality requirement, or an adapted student that fails to improve is an informative outcome. No such

outcome is asserted in this chapter. The proposed methods, source limitations and decision criteria provide the basis on which later measured evidence can support or reject the thesis’s claims.

Chapter 3

System design and formalization

3.1 Design principles

The system follows a reuse-first principle. Existing software supplies the agent loop, repository tools, local inference, provider API adaptation, training and benchmark execution. Custom components are introduced only for the research question: recording the decision state, applying a routing policy, measuring costs and joining outcomes to immutable task identifiers. This separation helps avoid comparing an extensively engineered portfolio system against an artificially weak baseline.

The proposed repository-level harness is mini-SWE-agent, subject to a pinned-version integration check. Its command-oriented interface allows the same repository interaction pattern to be used with multiple model providers. Ollama or llama.cpp is a candidate local deployment runtime; the actual supported model architecture, quantization, chat template and tool behavior must be verified together. AgentLab/BrowserGym is the browser-domain scaffold candidate. Managed endpoints or vLLM can supply remote configurations, including small models and compatible customer adapters. Transformers, PEFT and TRL provide the student-training stack. Choosing these components is a design decision, not a claim that all combinations have passed compatibility tests.

The experiment separates four concerns: the policy determines which model to use; the model proposes an action; the harness determines whether and how that action may execute; and the evaluator assesses the final artifact in an independent environment. A model switch does not grant new tool permissions. Tests used by the agent for development are distinguished from hidden benchmark checks used for final scoring.

3.2 Model registry, state and eligible actions

The model registry records each candidate’s domain and modality support, checkpoint or service revision, license, tokenizer/template, precision, inference settings, endpoint, measured resource requirements and compatible adapter identity. It distinguishes capabilities documented by a provider from capabilities verified in the common harness. A customer preference for a provider, an offline-only mode or a device memory limit changes the eligible set before the router scores it.

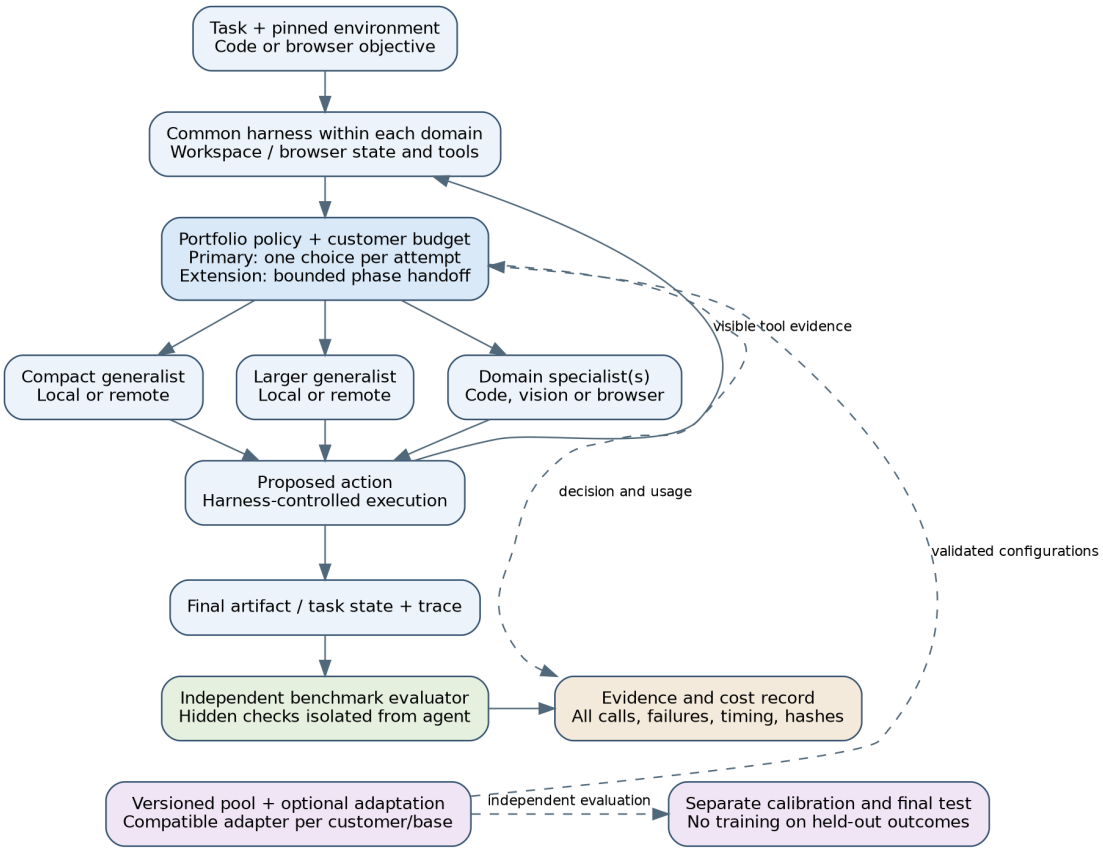


Figure 3.1: Proposed architecture and evidence flow. Arrows indicate interfaces, not measured performance.

Let task i describe an issue or browser objective in a pinned environment. At decision step t, the observable state is

$$s_{it} = (x_i, h_{it}, w_{it}, b_{it}, d_i, m_{t-1}, e),$$

where x_i contains permitted task context, h_{it} is interaction history, w_{it} is the observable workspace or browser state, b_{it} is remaining budget, d_i records deployment and customer constraints, m_{t-1} identifies the current configuration, and e identifies the frozen portfolio epoch. The router receives a declared feature function $\phi(s_{it})$. It cannot access hidden evaluation tests, reference patches, future tool results or eventual success labels.

The eligible model configurations are $\mathcal{M}_e(s) \subseteq \{m_1, \dots, m_K\}$. The action set contains those configurations and a stop action. Verification may be an explicit action in a secondary sequential experiment and carries its own resource cost. The initial treatment selects one model configuration for the complete attempt. Parallel voting, multiple simultaneous agents and weight merging are separate interventions, not necessary features of this portfolio.

A policy $\pi(m \mid s, \lambda)$ chooses an eligible action subject to permissions, context capacity and budget. It records whether a candidate was excluded by capability, price, memory, availability or customer policy. No candidate fitting the remaining constraints yields a recorded unresolved or budget-exhausted outcome. An undocumented fallback must not silently change the experimental condition.

3.3 Expected utility and customer price sensitivity

For configuration m, let $p_m(s)$ be the probability of successful task completion under a bounded continuation, $c_m(s)$ its expected remaining economic cost, and $\ell_m(s)$ its expected remaining latency. The quality definition belongs to the domain evaluator. A simple proposed decision rule is

$$m^*(s) = \operatorname*{arg\,max}_{m \in \mathcal{M}_e(s)} \left[\hat{p}_m(s) - \lambda \hat{c}_m(s) - \eta \hat{\ell}_m(s) - \hat{\kappa}(m_{t-1}, m, s) \right].$$

Here λ expresses the cost penalty, η expresses latency preference, and κ expresses any additional handoff penalty in the same utility units. Monetary and time components of handoff are included either in c_m, ℓ_m or in κ, never twice. Quality loss from summarization or state transfer should be reflected in continuation success estimates. These are quantities to measure and calibrate; the notation is not evidence that they are known accurately.

The product can expose a price-per-task budget or an accepted quality target rather than an uninterpretable numerical λ. Development data map that preference to a calibrated policy. Both an expected-spend target and a hard attempt cap are recorded: an expectation alone is not a guarantee about any individual bill. Higher price sensitivity may favor a hosted small model, a local model or an inexpensive specialist, depending on the complete costs. Neither size nor location fixes the choice.

The earlier two-model rule is a special case. With candidates L and R and equal nonfinancial constraints, choosing R requires its expected success improvement

over L to justify its incremental cost. A local-failure score $1 - \hat{p}_L$ is an informative comparator, but it does not identify which of several alternatives can help. For a larger pool, success and cost estimates must be assessed per candidate and on the tasks where the policy selects it.

Task features can include permitted issue/context embeddings, programming languages, frontend/backend indicators, browser modality and resource availability. These labels can overlap, and a task can require several capabilities. A domain label is a useful low-cost baseline rather than a reason to assume one model is universally best for that domain. Features requiring a model call or repository analysis incur acquisition cost.

Calibration is an empirical relationship between predictions and outcomes. Verbal confidence, token likelihood and self-evaluation are candidate features, not correctness guarantees. Outcome-informed selection over all candidates measures diagnostic headroom; it does not demonstrate that the deployed router reaches the union of their capabilities. New models acquire development evidence and a versioned router/calibration update before joining a test epoch. This does not require retraining every generator, and retraining the router alone does not prove that a new candidate is useful.

3.4 Sequential delegation and state continuity

A sequential handoff differs from task-level selection. It first pays for work by one configuration, observes permitted evidence and then decides whether another should continue. If the first attempt changes files, the continuation receives a well-defined state: base commit, current patch, task objective, executed commands, outcomes and remaining budget. A browser handoff additionally preserves the authenticated session, tabs, navigation state and side-effect record through the shared browser environment; session state is distinct from a model cache. Unfinished tools are completed or cancelled before switching. The system does not attempt to transfer incompatible key–value caches or opaque provider reasoning representations.

All work before handoff remains part of the portfolio's cost. A continuation can require extra context and have different success probability from a fresh attempt. The study distinguishes $p_m(x)$ from $p_m(s_{it})$ after another configuration has acted. Substituting an archived standalone outcome for that continuation would erase a central system effect. The default is to retain a model for an entire attempt; phase-level switching and a minimum net-benefit margin are tested before unrestricted per-call switching. The adaptation and switching chapter defines cache and adapter compatibility boundaries.

For paired comparisons, every experimental condition starts from the same immutable repository snapshot. Workspaces, generated files and cached agent conclusions are isolated by task, seed and condition. Dependency caches may be shared only when they do not expose solutions or mutable task state. Replaying a tool event is not equivalent to re-executing it: external side effects, generated dependencies and process state require explicit treatment.

3.5 Confidence-scheduled verification as an analogy

DeepInfer provides a concrete implementation example of using learned confidence estimates to allocate inference computation. Its DSpark mechanism predicts draft-token acceptance and schedules verification using acceptance estimates and runtime cost information. This motivates a general design principle: spend computation where it is predicted to produce useful progress, and compare predicted benefit with observed behavior.

The analogy has limits. Token-level speculative decoding involves a particular relationship between a draft distribution and target verification. A task router that accepts a local patch does not inherit that algorithm's distribution-preservation properties. Repository tests are incomplete, remote reviews can be wrong, and task failure events are correlated. Multiplying arbitrary subtask confidence scores as if they were independent is unjustified. The proposed system uses direct task/continuation outcome labels and reports uncertainty rather than importing a token-level guarantee.

3.6 Student learning mechanisms

The adaptation study begins with sequence distillation. For training task i, a teacher generates an observable solution or trajectory, which is retained only if it passes the declared verification procedure. The student is trained on the resulting target sequence y_i^T:

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\sum_i \sum_{t \in \mathcal{T}_i} \log p_\theta(y_{it}^T \mid x_i, y_{i,<t}^T),$$

where $\mathcal{T}_i$ contains completion tokens included in the loss. Prompt tokens are masked. LoRA changes which parameters are optimized; it does not determine whether the targets are correct or representative. Complete examples that exceed the declared context limit are explicitly handled instead of silently truncating a solution and treating the remainder as valid supervision.

Verification filtering creates selection effects. Tasks with no verified teacher solution disappear from the training set, while the evaluation corpus must retain its prespecified eligibility rules. A student may consequently receive a distribution biased toward tasks the teacher already solves. Counts, task classes and exclusion reasons are recorded. A token- and compute-matched adaptation control is needed to distinguish teacher-data quality from the mere effect of extra training.

A token-distribution arm has a different objective. With aligned teacher/student tokenization, an example formulation is

$$\mathcal{L}_{\mathrm{KD}} = \alpha \mathcal{L}_{\mathrm{SFT}} + (1-\alpha)\tau^2 \sum_t D_{\mathrm{KL}}(p_T^\tau(\cdot \mid c_t) \,\|\, p_\theta^\tau(\cdot \mid c_t)) \,.$$

Temperature τ, mixture coefficient α, context c_t, vocabulary mapping and teacher precision must be specified. Different Qwen and DeepSeek tokenizers cannot be assumed to share token events. A same-family teacher/student pair is a practical controlled case, but still requires exact vocabulary, special-token and template

checks. A provider's limited top-k log probabilities are not the full teacher distribution; treating the omitted probability mass as zero changes the objective and requires its own method and ablation.

RLAIF describes the use of AI-generated feedback. It is distinct from using executable tests as a verifiable reward. DPO on teacher-ranked pairs is one possible preference-learning arm; DPO on pass/fail pairs is verifier-based preference learning. Online reinforcement learning adds generation and reward-computation costs and is a separate extension. The experiment does not relabel ordinary supervised fine-tuning as RLAIF.

3.7 Adaptation and routing as separate interventions

An adapted portfolio member changes the distribution of errors on which the router was trained. The experiment evaluates both a frozen router with the adapted student and a newly calibrated router fitted without final-test labels. This forms a factorial distinction between student capability and routing adaptation. Otherwise, an apparent gain in inexpensive-model coverage could be attributed incorrectly to the generator when it arose from changed allocation. A customer may share verified examples across compatible training pipelines while retaining a separate adapter artifact for each selected base revision.

Model/adapter releases are immutable artifacts with hashes, base revision, tokenizer revision and training provenance. The exact deployed artifact—including quantization or adapter merging—is reevaluated. Training-time loss alone is not a software-engineering outcome, and improvements before export do not guarantee the same behavior in a local runtime.

3.8 Cost model

For model call j, API expense is recorded using the provider's billed categories:

$$C_j^{\text{API}} = \frac{r_u n_u + r_c n_c + r_o n_o + r_w n_w}{10^6} + f_j,$$

where n_u, n_c, n_o, n_w represent uncached input, cached input, output and any separately billed cache-write units, with their respective rates. A category is omitted or reformulated when a provider bills differently. Reasoning/thinking tokens are included according to the actual invoice definition; they must not be counted twice as both visible output and an independent category. Tool, search, batch, regional and context-tier charges are stored explicitly when applicable.

The complete task cost includes all model calls, routing, verification, execution and relevant allocations:

$$C_i = \sum_{j \in i} C_j^{\text{model}} + C_i^{\text{router}} + C_i^{\text{sandbox}} + C_i^{\text{verification}} + C_i^{\text{storage/network}} + C_i^{\text{allocated}}.$$

For self-hosted models, GPU rental is charged for provisioned time, including loading, idle periods and shutdown delay. Local operation is reported under two

accounting views: marginal cash expenditure and total economic cost including energy and hardware allocation. A zero API rate is therefore not represented as zero total cost. Cost per resolved issue has the total cost of all assigned attempts, including failures, in its numerator and the number of resolved issues in its denominator. Under repeated sampling, the numerator and denominator use the same registered aggregation. Zero resolved issues makes this ratio undefined; unsuccessful runs cannot be removed from the invoice.

Distillation adds an amortization question. If training and data generation cost C_{adapt} and the adapted system saves an expected s per task at an acceptable quality level, an illustrative break-even task count is $N = C_{\text{adapt}}/s$ for $s > 0$. No break-even claim is valid when quality is not maintained or the estimated saving is nonpositive. Estimates must be accompanied by workload assumptions and uncertainty rather than presented as a guaranteed product margin.

3.9 Evidence record

Every run records task and repository identifiers, base commit, dataset/harness revisions, container image digest, condition, model/adapter revision, decoding settings, seed where supported, resource limits, prompts/tool traces under the retention policy, output patch, evaluator logs and billed usage. Cached and uncached scenarios are identified separately. A run manifest links these records through hashes and identifies whether a stage was prepared, started, completed, failed or unrun.

This evidence structure serves both scientific reproducibility and task continuity. Jira tickets can index work and Confluence can host the manuscript, but neither substitutes for run manifests and versioned artifacts. If external tools are unavailable, local state records remain explicitly pending synchronization and must not invent issue keys, page IDs or completed runs.

Chapter 4

Model and price landscape

This chapter freezes the **model-selection evidence on 25 October 2026**. It contains published vendor claims and a dated price catalog, not results from the TTGL experiment. “Latest” means the latest relevant release found in the inspected official repositories and model cards; this is not a claim of exhaustive market coverage. The accompanying model catalog records exact identifiers, model revisions, licenses, service names, units, and evidence status. The source register preserves retrieval timestamps, content hashes, revision-pinned URLs, selected source excerpts, and unsuccessful pricing requests.

4.1 Candidate selection

The current Qwen release catalog lists Qwen3.8-27B, released on 14 August 2026, and Qwen3.8-2.4T-A95B, released on 12 August. Its most recent sub-10B dense releases are **Qwen3.5-0.8B, 2B, 4B, and 9B**, released on 2 March. Consequently, the initial development shortlist includes **Qwen3.5-4B**, **Qwen3.5-9B** for a capacity comparison, and **Qwen3.8-27B** as a potential self-hosted teacher or portfolio member. These roles are hypotheses to test, not evidence that one fixed size pair is optimal. Calling a small local checkpoint “Qwen3.8” would misidentify the verified release. Qwen2.5 remains useful for reproducing the existing implementation, but it is a historical baseline rather than a current-generation claim.[1]

The ten candidates below form the initial software-engineering market register. They cover local inference, remotely hosted open weights, inexpensive closed APIs, and premium closed APIs. The browser-domain chapter adds modality-specific evidence and candidate roles. This is neither a fixed two-model architecture nor a final best-model ranking: several small or large configurations may be eligible, and pool membership is selected on development data. Candidate status does not imply that TTGL already supports the model or that an account can access it.

Candidate	Proposed role	Parameters: total / active	Weight access and license
Qwen3.5-4B	Primary local student	4B / dense	Open weights, Apache-2.0

[1] Qwen official release catalog. Individual weight-card revisions and licenses are recorded in `model-catalog.csv` and `market-sources.json`.

Candidate	Proposed role	Parameters: total / active	Weight access and license
Qwen3.5-9B	Local capacity ablation	9B / dense	Open weights, Apache-2.0
Qwen3.8-27B	Self-hosted teacher or remote baseline	27B / dense	Open weights, Apache-2.0
DeepSeek-V4.1-Flash-1017	Remote teacher candidate	Base family: 552B backbone; 8B prefill / 16B decode active	Open weights, MIT
GLM-5.3-Flash	Low-price remote baseline	320B / 18B	Open weights, MIT
Kimi K3	Remote frontier reference	2.8T / 104B	Open weights, custom Kimi K3 License
GPT-6.1 Sol	Premium closed baseline	Undisclosed	Closed weights
GPT-6 Luna	Inexpensive closed baseline	Undisclosed	Closed weights
Claude Sonnet 5.5	Premium closed baseline	Undisclosed	Closed weights
Claude Opus 5.5	Higher-price closed reference	Undisclosed	Closed weights

The official OpenAI and Anthropic SDK model catalogs identify the closed-model API names `gpt-6.1-sol`, `gpt-6-luna`, `claude-sonnet-5-5`, and `claude-opus-5-5`. An SDK entry establishes an identifier, not an immutable weight revision, capability guarantee, or account entitlement. Freeze the returned serving revision and all provider settings at execution; do not silently roll an alias forward midway through a comparison.[2]

Open weights and open-source software are different properties. The Qwen licenses above were checked in the individual weight repositories. GLM-5.3-Flash weights use MIT even though its umbrella GLM-5 code repository uses Apache-2.0. Kimi K3 applies its custom license to both code and weights: among other provisions, it requires a separate commercial agreement for a model-as-a-service business above the specified aggregate revenue threshold and branding for certain very large services. The exact license includes exceptions and definitions; it must be read for the intended deployment. A permissive harness or SDK license does not license the model, and a model-weight license does not establish hosted-API output-training rights.[3]

The parameter distinction materially changes feasibility. GLM's 18B active parameters do not mean only 18B weights need storage. Even an idealized 4-bit representation of its 320B total parameters requires approximately 160 GB before scales, activations, KV cache, and runtime overhead. Kimi's 2.8T parameters require approximately 1.4 TB at that idealized precision. DeepSeek's base card specifies a 552B backbone and separately describes 196B Engram conditional memory; the inspected wording does not establish an unambiguous combined total, so this chapter does not add them. The backbone alone is already too large for full residency on one 80 GB GPU. Qwen3.8-27B has approximately 54 GB of

[2] OpenAI SDK model identifiers; Anthropic SDK model identifiers. Retrieved 25 October 2026.

[3] Qwen 4B weight license, 9B weight license, and 27B weight license; GLM-5.3-Flash weight license; Kimi K3 license; DeepSeek-1017 license statement.

raw BF16 weights, making a memory-profiled single-GPU inference trial plausible; that arithmetic is not proof of long-context or training feasibility.[4]

4.2 Price comparison and what it measures

All direct-API rates in the following table are provisional secondary observations, not vendor-verified quotes. They come from one revision-pinned LiteLLM community catalog snapshot. Direct OpenAI, Anthropic, DeepSeek, Nebius, and Z.ai pricing pages were requested and returned proxy `403` errors, including a later recheck after model-card access succeeded. Successful Hugging Face retrieval therefore does not establish access to those billing sources. The exact failures are recorded rather than replaced with guessed prices.[5]

Rates are **USD per one million text tokens**, in the order **uncached input / cache-read input / output**, for the catalog's ordinary base tier. They exclude cache creation, long-context multipliers, batch discounts, priority or fast modes, regional adjustments, tool charges, taxes, and subscription conditions. Missing cache prices are unknown, not zero.

Model / priced service	Input	Cache read	Output	Qualification
Qwen3.5-4B / local	—	—	—	No API token bill; hardware, energy, and training still cost money
Qwen3.5-9B / local	—	—	—	Same accounting boundary
Qwen3.8-27B / Nebius Token Factory	$0.45	Unknown	$3.00	Third-party service; not a Runpod rental quote
DeepSeek-V4.1-Flash / Tencent Cloud	$0.30	$0.006	$1.20	Alias-to-1017 mapping unverified; not direct DeepSeek pricing
GLM-5.3-Flash / Z.ai	$0.15	$0.03	$0.50	Base catalog rate
Kimi K3 / Moonshot	$3.00	$0.30	$15.00	Base catalog rate
GPT-6.1 Sol / OpenAI	$2.00	$0.10	$10.00	Standard tier, input at most 272K tokens
GPT-6 Luna / OpenAI	$0.10	$0.01	$0.50	Standard tier, input at most 272K tokens
Claude Sonnet 5.5 / Anthropic	$2.00	$0.10	$10.00	Standard text tier; separate cache-write charges
Claude Opus 5.5 / Anthropic	$4.00	$0.20	$20.00	Standard text tier; separate cache-write charges

An independently retrieved **primary GitHub Copilot pricing file** lists the same four closed-model base input, cache-read, and output rates. This corroborates those numbers for the Copilot channel only; it does not certify the direct-provider API contract. Its service plan, credit accounting, and model availability are separate. Neither source establishes a free API tier or the price of an available 80 GB Runpod instance.[6]

[4] GLM official model table; Kimi K3 architecture; DeepSeek base family architecture.

[5] Secondary source: LiteLLM price catalog, commit dddbaa8, retrieved 25 October 2026. Exact catalog keys, raw selected entries, and the catalog's cited upstream URLs are preserved in `market-sources.json`. Upstream direct API rates remain unverified.

[6] Primary source for its own service channel: GitHub Copilot model prices, retrieved 25 October 2026. This is not a direct-provider API quote.

This price table motivates inclusion of an inexpensive closed baseline: a portfolio system must compete with an already cheap API, not only a premium teacher. It does not establish cost effectiveness. A task may use many model calls, cached and uncached context, reasoning tokens, failed tools, and retries. The experiment must compute total inference and tool cost per attempted task, total cost per resolved issue, and wall-clock latency from actual traces. Local cost should be reported both as marginal energy plus incremental infrastructure and under an explicit amortized hardware scenario. Report distillation/training expenditure separately and amortize it over declared future workload sizes; a “free local” product tier is a billing choice, not zero economic cost.

4.3 Published accuracy evidence: contextual, not a leaderboard

Published scores establish that candidate models have been evaluated on relevant tasks. They cannot supply the missing TTGL comparison. Every score below is a **vendor-reported claim** from a primary artifact; none has been reproduced here, and none is a TTGL outcome. A blank latest-model score means insufficient inspected evidence, not zero capability.

Model	Published claim	Harness and limits disclosed by that source
Qwen3.5-4B	LiveCodeBench v6: 55.8	Coding-problem benchmark, not repository-level SWE; exact task count and harness not stated in the inspected card
Qwen3.5-9B	LiveCodeBench v6: 65.6	Same qualification; quantized local deployment must be measured separately
Qwen3.8-27B	DeepSWE v1.1: 42.2; SWE-bench Pro: 61.7	Claude Code; temperature 1.0, top-p 0.95, 256K context; Pro uses vendor-corrected tasks, not an unchanged official task set
DeepSeek-V4.1-Flash-1017	DeepSWE v1.1: 77.0	Mini-SWE-V2; task count, exact harness commit, and benchmark-specific sampling budget not stated in the inspected update card
GLM-5.3-Flash	DeepSWE v1.1: 63.4	mini-swe-agent, temperature 0.95, top-p 1.0, 400K context, six-hour timeout; task count and exact harness commit unspecified

Model	Published claim	Harness and limits disclosed by that source
Kimi K3	DeepSWE v1.1: 67.3	README footnote attributes this to mini-SWE-agent; main-table 67.5 uses Kimi Code; task count and exact harness commit unspecified
Four latest closed candidates above	Not established by inspected SDK/pricing sources	Catalog presence and prices provide no SWE accuracy evidence

The Qwen figures come from its exact model cards, the GLM figure from a pinned official chart plus its model-card evaluation footnote, and the Kimi and DeepSeek figures from their official release artifacts.[7] These are different benchmark families and operating points. LiveCodeBench is not SWE-bench Verified; SWE-bench Pro is not DeepSWE; corrected tasks are not the original evaluation set. Even the DeepSWE rows differ in context, timeout, harness, and version details. The table is deliberately **not sorted by score**, and no price–accuracy Pareto frontier is inferred from it.

The parent DeepSeek-V4.1-Flash card illustrates why harness control matters: it reports DeepSWE v1.1 scores ranging from 65.5 with OpenCode to 74.2 with mini-SWE for the same model, under a described eight samples per task, one-million-token context, and 500-step setting. The newer 1017 card reports 77.0 with Mini-SWE-V2; this cannot be treated as a clean model-only improvement over the older number. It also compares older closed releases, including GPT-5.6 Sol and Opus 5; those scores must not be reassigned to GPT-6.1 Sol or Opus 5.5.[8]

For the thesis, "software-engineering accuracy" should mean issue resolution by the benchmark's official executable evaluator. Compare always-local, always-remote, and routed systems on the **same held-out issues, harness revision, tools, prompts, retry policy, step limit, context policy, and task timeout**. A direct single completion without tools is a separate model-only task condition; it must not be presented as equivalent to an agent using shell access and tests. The closed and open models require this same evaluation before any capability or price–accuracy ranking is defensible.

4.4 Runtime readiness and the first bounded study

The verified releases are **research candidates, not deployment-ready TTGL configurations**. The existing Qwen2.5 training stack's Transformers 4.56.2 / TRL 0.23.1 pins do not establish support for Qwen3.5's hybrid architecture or Qwen3.8. The Qwen3.5-4B card recommends current frameworks, illustrates Transformers installed from `main` with serving support, SGLang from its Git repository, and vLLM with a nightly-wheel index. It specifies `qwen3` reasoning parsing and `qwen3_coder` tool parsing, and offers a language-model-only serving mode. It does **not** give a stable numeric minimum version that this chapter can certify. The Qwen3.8-27B

[7] Qwen model cards: 4B, 9B, 27B; GLM official benchmark chart and evaluation settings; Kimi README footnotes; DeepSeek-1017 model card.

[8] DeepSeek-V4.1-Flash base release card, especially "Performance across agent scaffolds"; this is separate from the 1017 update card.

card likewise recommends current SGLang, vLLM, or TokenSpeed recipes without specifying a numeric minimum in the inspected text.[9]

Resolve compatible released versions or immutable commits in a new lockfile; verify loading, tool calls, loss masking, one optimizer step, adapter save/reload, and evaluation on an excluded smoke-test task before a paid run. Serving support is not proof of fine-tuning or TRL compatibility. Keep model revision, tokenizer/template revision, quantizer, engine build, precision, reasoning-effort setting, and maximum context in the run manifest. For local Qwen, evaluate the same quantized deployment that the product intends to ship.

DeepInfer is valuable related work, but its current README targets **NVIDIA SM100 or Ascend 950**, requires Linux, GCC at least 13, Python 3.12, and CUDA 13 or CANN 9.2, and states that a complete service still needs KV-cache management, disaggregation, and load balancing. Its example benchmarks use four SM100 GPUs for prefill and 64 for decode. This is not the first experiment's single A100/H100-class 80 GB deployment. Its default checkpoint is exactly DeepSeek-V4.1-Flash-1017. Use the model through a suitable remote service for the bounded pilot if exact revision and API behavior can be established; use a manageable Qwen teacher for direct access to full logits.[10]

With a US$100 total GPU budget, the ten-model catalog is a selection space rather than a commitment to an exhaustive training matrix. For the smallest integration pilot, prioritize one small candidate, one reference model and the restricted two-model combination. That pilot verifies infrastructure; the expanded portfolio claim requires at least three eligible configurations under the ensemble protocol and a separately costed experiment. Add the 9B capacity ablation only after a costed smoke run establishes feasibility. An inexpensive closed API is a useful later comparator, subject to a separate authorized API budget; the Runpod approval does not cover that spending. Keep broad frontier comparisons descriptive until funded comparable evaluations exist. Teacher API text or top-k log probabilities must not be described as full logits; exact-token distillation additionally requires tokenizer compatibility or an explicitly validated alignment method. All TTGL held-out software-engineering and browser accuracy, portfolio calibration, savings, and adaptation gains remain unmeasured. The task-economics chapter provides attributable historical dollars per task and separate current-rate arithmetic scenarios; neither fills that empirical gap.

[9] Qwen3.5-4B serving instructions and Qwen3.8-27B serving instructions. Framework recommendations are vendor documentation, not a completed TTGL compatibility test.

[10] DeepInfer README, retrieved 25 October 2026; content hash and retrieved source URL in `market-sources.json`. Hardware and complete-service limitations are explicitly documented by the project.

Chapter 5

Published open- and closed-weight software-engineering accuracy

A follow-up search on 25 October 2026 inspected the official SWE-bench site's complete published leaderboard and the complete, non-truncated file tree of its submission repository. The published board contained 180 Verified, 84 Lite, 24 Test, 22 Multimodal, and 13 Multilingual entries. No identities matching **GPT-6.1 Sol, Claude Sonnet 5.5, Claude Opus 5.5, or Gemini 3.1 Pro** appeared in the inspected leaderboard or submission paths. This documents a limitation of the inspected evidence, not proof that no evaluation exists anywhere. In particular, the Gemini 3 Pro entry identifies `gemini-3-pro-preview`; its score cannot be relabeled as Gemini 3.1 Pro.[1]

The benchmark does provide an attributable **historical comparison of open and closed models** using the same named harness release. The following records are dated **17 February 2026**, use **mini-SWE-agent 2.0.0**, and specify **one attempt per issue on SWE-bench Verified**. Each selected leaderboard entry contains 500 per-instance outcomes; summing its published resolution flags reproduces the reported numerator and percentage. This consistency check verifies transcription of published records; it is not a new model experiment or independent rerun of the benchmark.

Historical model	Weight access	Reasoning effort	Resolved / 500	Resolution rate
Claude Opus 4.6	Closed	Unspecified	378 / 500	75.6%
GPT-5.2	Closed	High	364 / 500	72.8%
Claude Sonnet 4.5 (`20250929`)	Closed	High	357 / 500	71.4%
GLM-5	Open	High	364 / 500	72.8%
Kimi K2.5	Open	High	354 / 500	70.8%
MiniMax M2.5	Open	High	379 / 500	75.8%

[1] Official SWE-bench leaderboard JSON, site revision `193160a463a435d05cf44a1fa9dc5eac832113c3`; official submission repository, revision `40f164d5b8f1d249bf95a6df8b74b577fd8e519d`. Retrieved 25 October 2026.

These are primary **benchmark-published submission records**, with per-instance details and linked trajectories, rather than scores copied from a competing vendor's comparison chart. Weight-access classifications and reasoning settings are those of the benchmark records. The selected records have `checked: null`; that must not be rewritten as an independently audited result. Shared task set and harness version improve comparability, but the inspected metadata does not establish identical sampling, context, tool budgets, serving revisions, or reasoning settings. Therefore this is historical descriptive evidence, not a causal ranking of model families, proof of equivalence, or evidence that an open model beats a current closed model. The single-issue difference between two records is not a statistical significance claim.[2]

The comparison supports a concrete research motivation: the benchmark's older open-weight and closed-weight systems both resolved substantial fractions of the same repository-level task set, making measured cost and routing behavior useful questions. It does **not** establish the accuracy of Qwen3.5-4B, Qwen3.8-27B, DeepSeek-V4.1-Flash-1017, GPT-6.1 Sol, or Claude 5.5. Historical resolution percentages must not be combined with current token prices to construct a purported current-market Pareto frontier. The thesis's current-model comparison still requires its preregistered common-harness evaluation.

A newer Google submission supplies another clearly identified contextual point: **Gemini 3.5 Flash with mini-SWE-agent 2.4.2**, dated **1 September 2026**, reports **359 / 500 = 71.8%** on SWE-bench Verified. Its README states that it used the bundled `swebench.yaml` unchanged, one attempt per instance, and that 441 instances were attempted while 59 produced no patch. Its denominator remains 500. The metadata specifies medium reasoning effort and harness commit `2f6637a2557da1f7b5f2b583ab7090a077bbeed2`. This submission exists in the experiment repository but was absent from the inspected generated website board, illustrating publication lag. It is a different model and harness from the historical table and is not evidence for Gemini 3.1 Pro or the latest Google release.[3]

The accuracy evidence register contains revision-pinned source URLs, SHA-256 hashes, selected raw submission metadata, exact record names, and the search scope. No TTGL accuracy or routing outcomes are introduced by this chapter.

[2] Pinned official submission metadata: Opus 4.6, GPT-5.2, Sonnet 4.5, GLM-5, Kimi K2.5, MiniMax M2.5.

[3] Gemini 3.5 Flash submission README and metadata.

Chapter 6

Task-level prices and portfolio economics

6.1 The unit of purchase is a completed attempt

A token tariff is an input to a cost model, not the price of repairing a repository or completing a browser workflow. One task can consume many prompts, tool results, screenshots, retries and model transitions. Two models offered at the same token rate can therefore have very different task costs. The study reports **US dollars per assigned task** and **US dollars per successfully completed task**, alongside the success rate and latency distribution. Task-specific costs are stratified by domain and difficulty proxies established before outcomes are observed.

For a collection of assigned tasks with total recorded costs C_i and success indicators Y_i, the two basic quantities are

$$\overline{C} = \frac{\sum_i C_i}{n}, \qquad C_{\text{success}} = \frac{\sum_i C_i}{\sum_i Y_i}.$$

All assigned attempts contribute to the numerator, including failed runs. The second ratio is undefined when no task succeeds. It is an aggregate acquisition cost per observed success, not the cost of repeatedly attempting any particular issue until it succeeds. That interpretation would require assumptions about repeated attempts, dependence and stopping that this calculation does not make. For repeated-seed evaluations, costs and successes use the same registered aggregation.

Three evidence classes are kept separate: historical benchmark-reported costs, illustrative arithmetic using stated token volumes, and future TTGL costs reconciled to usage and billing records. Only the third can establish the deployed system’s actual economics. The first two are useful for planning without inventing unobserved task outcomes.

6.2 Attributable historical cost per software-engineering task

The official SWE-bench website exposes an `instance_cost` field and labels it "Average cost per task instance in the benchmark." Its pinned leaderboard also supplies individual costs and resolution outcomes. For the six historical records below, summing all **500 per-instance costs** reproduces the reported total; dividing by 500 reproduces `instance_cost`. Resolution counts were checked against the same records. The table uses those reported costs and derives cost per resolved issue without changing historical prices.[1]

All six submissions are dated **17 February 2026**, use **mini-SWE-agent 2.0.0** on **SWE-bench Verified**, and specify one attempt per issue. Monetary columns are rounded to four decimals; the accompanying CSV preserves the source precision and calculation.

Historical model	Resolution	Reported $ / assigned task	Derived $ / resolved task
MiniMax M2.5, high effort	75.8%	$0.0733	$0.0967
Kimi K2.5, high effort	70.8%	$0.1466	$0.2070
GPT-5.2, high effort	72.8%	$0.4736	$0.6505
GLM-5, high effort	72.8%	$0.5344	$0.7341
Claude Opus 4.6, effort unspecified	75.6%	$0.5515	$0.7295
Claude Sonnet 4.5, high effort	71.4%	$0.6579	$0.9214

These figures are **benchmark-reported cost, not verified invoices or current service quotations**. The inspected records do not establish that repository containers, idle infrastructure, development, tax, storage or human effort are included. They must not be presented as full production cost. The entries also have `checked: null`, and shared harness/version metadata does not prove identical context, sampling, serving or tool budgets. No statistical ranking or current-model superiority claim follows from the table.

The useful observation is methodological: an existing benchmark can attach dollars to complete task attempts rather than only publishing per-token prices. That is the granularity required for this thesis. The manuscript's historical accuracy chapter explains the remaining comparability limits. These figures cannot be relabeled as the task prices of a successor checkpoint or combined with a current model's vendor accuracy score. The historical cost CSV links every row to its original submission identifier.

6.3 Illustrative prices for current catalog candidates

Actual task traces for the current candidates do not yet exist. The following table therefore computes **text-token charges for two assumed episodes**, using the dated secondary rates in the model catalog. It is a sensitivity calculation, not a

[1] SWE-bench website at revision `193160a463a435d05cf44a1fa9dc5eac832113c3`: leaderboard JSON, table rendering and average-cost label, and per-instance cost chart. Retrieved 25 October 2026. The selected submissions and hash are retained in `accuracy-context-sources.json`.

benchmark or a prediction of how many tokens a model will need. Counts are totals over all calls in an episode, not the size of one prompt:

- **Scenario A:** 20,000 uncached input tokens, no cache-read tokens and 4,000 output tokens.
- **Scenario B:** 100,000 uncached input tokens, 400,000 cache-read tokens and 20,000 output tokens.

Each provider's own tokenizer determines its bill. Equal token counts here provide a controlled arithmetic comparison; they do not imply equal text, reasoning, success or latency. Actual cache eligibility and the partition into billing categories must be measured. Scenario B assumes eligible already-cached reads and adds no separately billed cache creation. If that assumption does not match the provider, the scenario must be revised rather than treating cache creation as free. The table excludes images, browser execution, cache-write premiums, provider tools and other charges.

Candidate / priced service	Scenario A	Scenario B
Qwen3.8-27B / Nebius Token Factory	$0.0210	Unknown
DeepSeek-V4.1-Flash alias / Tencent Cloud	$0.0108	$0.0564
GLM-5.3-Flash / Z.ai	$0.0050	$0.0370
Kimi K3 / Moonshot	$0.1200	$0.7200
GPT-6.1 Sol / OpenAI	$0.0800	$0.4400
GPT-6 Luna / OpenAI	$0.0040	$0.0240
Claude Sonnet 5.5 / Anthropic	$0.0800	$0.4400
Claude Opus 5.5 / Anthropic	$0.1600	$0.8800

Prices retain the model catalog's **secondary-source, unverified direct-provider rate** status. The Tencent alias is not verified as the exact `1017` checkpoint. Qwen's Scenario B is unknown because the inspected catalog provides no cache-read rate; missing price is not zero. Local Qwen models are omitted from this API table because they require hardware accounting, not because their task cost is assumed zero. The arithmetic is reproduced in the scenario CSV. No cost-per-success column is supplied because contemporary task success is unmeasured.[2]

For browser tasks, image charges may depend on image size, tiling, rescaling or model-specific token conversion. Repeated screenshots and accessibility-tree observations can dominate context volume. The browser study consequently records billed image/text categories and environment time separately. A text-only scenario is not advertised as the price of a visual browsing task.

6.4 Local and hosted infrastructure costs

The economic boundary must match the deployment. A provider-hosted model incurs its charged API categories. A self-hosted model incurs provisioned compute, including model loading, idle capacity and shutdown delay. The same inference must not be charged twice by adding a hypothetical GPU allocation on top of

[2] Secondary LiteLLM price catalog, revision `dddbaa803d6e9fcc862ea7a5412f771cb1ae040e`, as recorded in `model-catalog.csv` and `market-sources.json`, reviewed 25 October 2026. The model-landscape chapter explains service identities, aliases, price units and unsuccessful direct-provider price verification.

a hosted API tariff that already pays for that inference. Separate browser or repository workers still have their own costs.

For a dedicated device or rental, a planning approximation is

$$C_{\text{compute/task}} \approx \frac{r_{\text{hour}}\, t_{\text{active}}}{3600u},$$

where t_{active} is seconds of task-attributed busy time and u is the assumed utilization of provisioned capacity. This is appropriate only when allocation and concurrency are defined consistently. It cannot substitute for measuring batched or overlapping workloads. As an **arithmetic example**, a hypothetical $2/hour service with 90 seconds of attributed busy time costs $0.05 per task at full utilization and $0.20 at 25% utilization. Neither the hourly price nor the runtime is a Runpod quote or measured model performance.

Measured reporting reconciles assigned task shares, idle capacity and setup costs with the total resource bill. A portfolio that keeps several large checkpoints resident may lose its apparent advantage through idle memory, replicas or queueing. Shared multi-adapter serving may reduce this overhead but still requires an explicit tenant and task allocation rule. Local-device reporting includes a marginal-cash view and an ownership/energy view; user hardware is not economically free simply because the application does not charge an API fee.

6.5 Adaptation, switching and the price-sensitivity policy

Portfolio inference is only part of lifecycle cost. Customer adaptation adds teacher-data generation, verification, training, adapter storage and validation. If the relevant total is F and a declared customer workload contains M future tasks, its amortized contribution is F/M. Both F and M are reported; the manuscript does not select a large hypothetical workload merely to make training appear cheap. With several student bases, each base's adapter and compatibility evaluation contribute to the customer's fixed cost. A shared data pipeline does not make all adapter training free.

A switch can add input reprocessing, weight or adapter loading, summaries and extra validation. Its cost belongs to the receiving task even when the switch fails. Cached-context savings require an actual eligible cache hit. A change in model or adapter may alter both output quality and cache validity, so a policy should compare the expected benefit of switching with the complete transition cost. Keeping a model for an episode or a coherent phase is a candidate policy, not an untested guarantee of lower total cost.

Price sensitivity is expressed through a task budget or a prespecified cost weight in the routing objective. For each eligible action, the router predicts success and **remaining total task cost**, rather than ranking model token tariffs. Its features can include domain, permitted repository/browser state, current model residency, remaining budget and customer policy. Frontend, backend and language labels are useful only if their predicted differences generalize to held-out tasks. A small remote specialist may be preferable to a larger local model; deployment location, capability, parameter size and price are distinct variables.

The evaluation reports paired success differences, mean task cost, cost per success, latency and upper cost quantiles at each registered operating point. Mean cost alone can conceal a minority of tasks that exhaust their budgets. Customer-facing estimates should therefore distinguish a quoted cap from an expected spend and the platform's internal cost from its selling price. An application may offer a free local tier, but that business decision does not change the study's hardware accounting. Actual closed-provider usage still requires its own authorized budget.

6.6 Reproduction and evidence boundary

`python scripts/build_task_price_table.py` regenerates both CSVs from the recorded source register and model catalog. Adding `--verify-upstream` downloads the revision-pinned public leaderboard, checks its SHA-256 and rechecks all 3,000 selected per-instance cost/outcome records. This is a source-consistency check and performs no model inference. The check record and economic source register identify the inputs and source code establishing the displayed cost semantics. Future empirical tables must replace assumptions with traces, reconciled charges and the declared failure accounting.

Chapter 7

Browser use and specialization across domains

7.1 What constitutes a separate domain

A useful ensemble can contain several language and vision-language models whose strengths overlap imperfectly. Model size, deployment location and task specialization are separate attributes. A small model can run on a remote GPU; a larger quantized model can run on a sufficiently capable desktop; a browser specialist can outperform a more general model on one interface while being unsuitable for repository repair. The study must discover these relationships through comparable outcomes rather than encode a universal small-to-large ranking.

Software engineering, browser automation and desktop operation require different environments and success criteria. Frontend programming means changing an application's code and verifying its behavior, potentially through browser tests. Browser use means operating an existing application through its exposed interface. Backend programming and programming language are useful software-engineering strata, but they do not by themselves define a distinct agent action space. Desktop use extends beyond browser tabs to operating-system applications and files. These distinctions prevent a strong visual grounding score from being presented as proof of effective frontend programming or reliable end-to-end browser automation.

Workload	Typical observable state	Primary outcome
Repository repair	Issue, files, visible test output	Benchmark-verified patch resolution
Frontend implementation or repair	Specification, source, rendered interface	Prespecified functional checks; visual fidelity separately
Browser task with structured observations	User goal, accessibility tree or DOM, browser state	Task-specific final-state or answer validation
Browser task with visual observations	User goal, screenshots, visible history	Same task validation under the registered visual interface
Desktop operation	Application screenshots, permitted file state	Desktop benchmark's execution-based validation

Routing should first restrict the candidate set to models compatible with the task's observation and action contract. A text-only backend is ineligible for a screenshot-only condition unless a separately declared perception component supplies a supported representation. That component's cost, errors and latency belong to the evaluated system. Conversely, supplying DOM element identifiers to a model assessed as a visual-only operator changes the treatment. The initial study uses one common observation/action contract within each comparison and treats alternative interfaces as separate ablations.

7.2 Reusing browser research infrastructure

BrowserGym and AgentLab are the preferred research components. BrowserGym supplies browser-task environments; AgentLab supplies experiment orchestration, model interfaces, traces and analysis. The canonical WebArena repository explicitly recommends this ecosystem for new experiments. This is stronger justification for reuse than recreating a browser agent loop solely to insert a router (BrowserGym contributors 2026; AgentLab contributors 2026; WebArena contributors 2026). The additional TTGL work is the multi-model decision hook, immutable model/policy manifest, cost reconciliation and joins between router decisions and validated outcomes.

The primary browser candidate is **WebArena-Verified**, subject to compatibility and environment checks. Its official project describes a version-controlled task set, deterministic evaluators operating on responses and captured network traces, and a 258-task Hard subset. AgentLab lists the full benchmark as 812 tasks. "Offline evaluation" means that captured runs can be graded without replaying a live website; it does not mean that an agent's interactive trajectory can be generated without an environment. Pinning the grader, dataset and trace schema together is essential (WebArena-Verified contributors 2026).

VisualWebArena supplies 910 visual web tasks and is a suitable second browser setting when screenshot understanding materially affects task success. It is not interchangeable with WebArena-Verified. Its documented task templates also make template-level leakage a concrete concern: the project provides human traces for one task from each of 233 template types. The thesis uses those public traces only as attributable background or development material, never as unseen final-test demonstrations (VisualWebArena contributors 2026).

WorkArena offers a complementary enterprise workflow distribution. Its current documentation describes 33 atomic task types with 19,912 possible instances, and WorkArena++ describes 682 compositional tasks. AgentLab separates 341 L2 and 341 L3 templates. Sampling many seeds from one template does not create equally many independent tasks or applications. WorkArena also requires access to ServiceNow instances through its documented access workflow; open-source harness code does not imply unrestricted, perpetually free hosted environments (WorkArena contributors 2026).

OSWorld-Verified is reserved for a desktop extension. AgentLab lists 369 OSWorld tasks; the upstream project specifically distinguishes its July 2025 Verified update from older OSWorld evaluation. Some tasks require external account configuration, and missing setup can cause failures. A browser-only claim should not use an aggregate desktop score as its direct outcome, and an old

OSWorld percentage cannot be relabeled as an OSWorld-Verified result (OSWorld contributors 2026).

The Browser Use library is a maintained product-integration candidate, with a documented local or cloud browser and replaceable model backend. Playwright supplies an existing cross-browser automation foundation. These tools can support later product deployment, while the primary research comparison retains its common BrowserGym contract (Browser Use contributors 2026b; Microsoft 2026). Replacing the harness for one favored model would confound the model-allocation intervention. A separate product-profile comparison can assess complete, model-specific systems if its interpretation is explicit.

Browser Use's benchmark illustrates why versions matter. Its current repository documents **BU Bench V2.1 with 200 tasks**, release v2.1.1, and September 2026 rubric revisions. The prominent model-versus-cost plot is explicitly a **historical 60-task comparison**, not a result on the current 200 tasks. The default benchmark uses weighted findings rubrics, and at least one task's revised contract makes earlier scores noncomparable. That corpus can become an external replication after evaluator and access review; it must not be pooled with deterministic binary browser success or used to claim a current-model frontier without matching the task revision (Browser Use contributors 2026a).

7.3 Candidate models and limits of published evidence

The dated catalog should include both general multimodal models and released GUI specialists. The inspected Qwen3.5-4B, Qwen3.5-9B and Qwen3.8-27B cards all identify image-text-to-text checkpoints. They therefore deserve browser compatibility checks alongside their software-engineering checks; their role need not be limited to one domain. The UI-TARS repository provides a released UI-TARS-1.5-7B checkpoint and announces a later UI-TARS-2 technical report. Neither that announcement nor the inspected official model listing establishes downloadable UI-TARS-2 weights. The released 7B model remains an eligible historical specialist comparator rather than being mislabeled the latest reported system (ByteDance 2026).

The following figures establish source-attributed candidate relevance. They are **vendor-reported contextual claims**, not a common-harness ranking, TTGL outcomes or estimates of browser-task cost.

Candidate	Published contextual claim	Interpretation limit
Qwen3.5-4B / 9B	OSWorld-Verified: 35.6 / 41.8	Desktop aggregate; complete comparable run protocol not established by the inspected card
Qwen3.8-27B	WebArena-Verified: 64.8; OSWorld-Verified: 84.3	Browser score uses the official grader under an OSWorld scaffold; exact run configuration must be reconciled
UI-TARS-1.5-7B	OSWorld: 27.5; ScreenSpotPro: 49.6	Historical desktop and grounding metrics; neither is current browser-task success

The browser source register records exact card revisions, hashes and interpretation

limits. Qwen's 4B/9B values were checked against the model-column order in their shared comparison table. UI-TARS requires particular care: the same 7B card contains headline scores for a broader "UI-TARS-1.5" system, then a scale comparison that explicitly gives lower values for the released 7B checkpoint. Assigning the headline system's results to the downloadable checkpoint would produce a false small-model advantage.[1]

Browser Use also documents a hosted specialist identifier, `bu-2-0`. The inspected documentation establishes an API candidate, not downloadable weights or unrestricted local inference. It can be an optional service comparator after pricing and access are established, but it does not enter an "open weights only" frontier merely because its client library is open source. Similarly, the inspected Qwen3-VL release history provides useful older visual baselines; those releases are not substituted for the newer unified Qwen cards without stating the temporal distinction.

Candidate admission therefore records checkpoint or service revision, license, modality, action compatibility, precision, context limits, device requirements and measured resource use. It does not assume that an accessible model can be loaded by the existing training stack. New models enter a new catalog version after development compatibility tests, with the router recalibrated on new development outcomes. Test labels from the prior release are not repeatedly reused as a model-selection service.

7.4 Held-out specialization and task features

The cross-domain experiment asks whether learned allocation improves over the best affordable fixed model and simpler domain-specific rules. A router that sends all browser tasks to a designated browser model may already capture much of the benefit. That domain-selector baseline is therefore necessary alongside random allocation, cost-only selection and the best development-selected single model. Model count is an experimental factor; adding candidates can increase discovery cost, serving overhead and overfitting as well as improve coverage.

Within software engineering, prespecified slices include frontend, backend, full-stack, tests, build/configuration and documentation, plus programming language. These labels can overlap. Labels come from a documented annotation rule using the issue and permitted base-repository information before outcome inspection. A reference patch's file types or changed-line count cannot become a router feature. The initial Python-heavy SWE-bench study cannot by itself establish performance across programming languages. Multilingual claims require an appropriate repository corpus, with language and repository effects reported separately.

SWE-bench Multilingual supplies a concrete external option. Its pinned official dataset card describes 300 test issue–pull-request pairs across nine programming languages and 41 repositories, with unit-test evaluation. The languages are Ruby, Rust, PHP, Java, Go, C, JavaScript, TypeScript and C++. Their sample sizes differ substantially, so the report gives per-language denominators and uncertainty rather than ranking small slices by their raw percentages. The

[1] Pinned official model cards: Qwen3.5-4B, Qwen3.5-9B, Qwen3.8-27B, and UI-TARS-1.5-7B. Retrieved 25 October 2026. The register separates desktop, browser and single-step grounding claims.

dataset's `FAIL_TO_PASS` and `PASS_TO_PASS` fields are lists rather than JSON-encoded strings; importing it through existing SWE tooling requires a schema and harness compatibility check. This is a separate held-out replication, not additional training data obtained by opening its test labels (SWE-bench contributors 2026).

For a narrower visual frontend task, **Design2Code** provides 484 screenshot/HTML webpage pairs with existing generation and automatic-evaluation code. It is a feasible external test of screenshot-to-code rendering, not a benchmark of operating a browser or implementing a complete application workflow. Visual fidelity, valid rendering and functional user-flow tests are distinct outcomes; the first two cannot stand in for the third. The reference HTML remains hidden from generation and routing, and pages sharing templates or site design should remain in the same split. The existing benchmark is reused with its specified visual metrics instead of inventing a generic "frontend accuracy" score (Stanford SALT lab 2026).

Within browser use, relevant features include the requested operation, site family, need for visual grounding, number of tabs, visible page complexity, expected authentication requirements, context size and available device capacity. Features obtained by opening a page or rendering a screenshot incur acquisition cost. The router cannot consult hidden rubric items, evaluator outputs or a task identifier encoding the benchmark answer. Customer identity can select a permitted adapter or deployment policy, but customer-specific access does not justify leaking private task content into a shared training set.

Splits are constructed by **template and site/application family**, not only by random instance ID. Near-duplicate instructions and generated variations stay together. One evaluation measures unseen task instances on familiar sites; another, where the corpus permits, holds out entire sites or applications. For a benchmark with few sites, uncertainty about new-site transfer remains large even with hundreds of tasks. WorkArena seed variation supports within-template robustness, while held-out templates test a different generalization claim. Each is named explicitly.

All eligible models use the same assigned tasks, initial state and registered action budgets within a domain. AgentLab documents task dependencies and environment-reset requirements: one task can alter a website so that another becomes impossible. The experiment must honor those dependencies or provide independently reset instances. A model's previous failed task cannot leave a corrupted environment for a competitor. Environment snapshots, reset outcomes, browser version, viewport, observation extraction and account fixtures belong in the manifest alongside model and adapter versions.

7.5 Measuring task value and cost

For domain d, report the mean cost per assigned attempt, $\sum_i C_{id}/n_d$, and the cost per verified success, $\sum_i C_{id} / \sum_i Y_{id}$. The numerator includes failed attempts, retries, router inference, model calls, visual preprocessing, browser execution and environment-reset overhead under the declared accounting boundary. Zero successes produces an undefined success-cost ratio. This ratio differs from the mean cost of successful traces alone, which would hide the price paid for failures.

Browser measurements require more than text token counts. Record screenshots

and resolution, the provider's billed image or vision-token categories, browser wall time, network charges where applicable, page loading, tool failures, model-loading delay and state reconstruction. A GPU-backed vision model's image encoder can affect memory and latency even when its language component resembles an existing text model. Measure local and remote deployment configurations separately rather than inferring one from the other.

The Browser Use README lists cloud-browser infrastructure at **US$0.02 per browser-hour plus network traffic**. At that rate, five provisioned minutes contribute about US$0.00167 before network and other charges. This is arithmetic applied to a dated vendor README observation, not a measured task price or a complete hosted-agent quote. Model calls, judge costs, retries and minimum billing terms remain separate. The execution plan must reconcile the actual service contract before using this figure in a purchasing decision (Browser Use contributors 2026b).

Binary task success, grounding accuracy and weighted rubric scores are reported on separate axes. A portfolio summary can weight domain-specific success rates by a deployment mixture fixed before final testing, with sensitivity to alternative mixtures. It cannot average an OSWorld percentage, a Browser Use weighted rubric and a SWE-bench percentage and call the result universal accuracy. Report domain-specific frontiers first, followed by any explicitly defined mixture objective and its weights.

7.6 Handoffs, adaptation and staged execution

Browser state and model state have different lifetimes. A handoff can preserve the same browser process, tabs, cookies and application state while replacing the model backend. The successor still needs a valid current observation and a bounded record of actions, results and unresolved goals. Its KV cache generally cannot be reused from an unrelated model. Re-rendering observations and reprocessing history add cost; summarization may omit decisive information. These are measurable transfer effects, not reasons to assume either that switching is free or that it always destroys progress.

The initial browser treatment chooses a model for the whole task. A secondary checkpoint policy can switch after a registered failure, uncertainty trigger or subgoal boundary. It uses fresh continuation runs from valid browser state, not archived independent trajectories spliced together. Log the reason for switching, state-transfer payload, reload/prefill time, additional calls and final outcome. A fixed switch limit and an estimated minimum benefit threshold constrain repeated oscillation between models.

Verified browser demonstrations can later train model-specific adapters or a small local model, using development tasks only. Browser trajectories contain sensitive state beyond visible prose, including authentication material and personal records; sanitized observation/action data and explicit tenant boundaries are part of the training contract. Adapter training is evaluated separately from router recalibration. A stronger router alone can shift traffic without improving any underlying model, while an improved specialist can still be misallocated by a stale router.

Execution remains staged. First validate one browser environment and its

evaluator on excluded development tasks, then test a small compatible candidate set before selecting a funded paired pilot. A broad SWE/browser/desktop matrix is not covered by the existing US$100 GPU feasibility allocation. The evidence needed for a cross-domain claim includes completed within-domain controls, measured task costs, held-out specialization, and uncertainty about the deployed task mixture. Until those records exist, the browser extension is a specified experiment and implementation path, not an achieved ensemble frontier.

Chapter 8

Customer adaptation, model placement and switching costs

8.1 The deployed expert is more than a model name

A portfolio expert is a complete deployment configuration: a base checkpoint, optional customer or domain adapter, tokenizer and prompt template, numerical precision, serving engine, placement and inference limits. A 4B model on a desktop, its quantized cloud deployment, and its customer-adapted version need not have identical accuracy or latency. They must have separate configuration identifiers even when they share a model-family name. Parameter count is therefore a candidate attribute, not a sufficient routing rule. A smaller browser specialist may outperform a larger general model on a particular interaction interface; that is a hypothesis to test under the common interface, not a consequence of specialization alone.

Two allocation decisions occur at different times. Deployment planning selects which base models to keep resident, where to serve them, and which adapters to retain near the accelerator. The request policy selects among the configurations that are currently available. A request router cannot obtain an unprovisioned model at the same cost and latency as a warm resident model. Conversely, keeping every candidate loaded can make an apparently inexpensive portfolio expensive through idle capacity. The experimental manifest records both decisions, and the cost model includes the resulting allocation and queueing effects.

8.2 What can be shared across customer adaptations

Ordinary LoRA learns a low-rank update to particular matrices of a particular base model (E. J. Hu et al. 2021). For a layer with weight $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, the adapted weight can be written

$$W' = W + \frac{\alpha}{r} BA, \qquad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}.$$

The dimensions, target-layer names and learned representation determine where this update applies. An adapter for one 4B model cannot generally be attached to an unrelated 9B model. Even matching dimensions do not establish semantic

compatibility: the underlying weights, representations and training history can differ. A new revision of the same named base is also not presumed equivalent. Transfer or adapter-conversion methods would be separate treatments with their own evidence, rather than an implicit feature of ordinary LoRA.

The practical shared object is a **customer adaptation package** containing approved training examples, provenance, evaluation cases, preferences and a mapping from eligible base revisions to their separately trained adapters. The package is shared at the product level; the tensor updates remain specific to their validated bases. PEFT's checkpoint format records the base identifier, revision and target modules, while its hotswap mechanism replaces adapter tensors inside an existing base model. Hotswapping is useful for compatible adapters on that base; it does not establish portability across arbitrary architectures.[1]

A minimum adapter manifest contains the customer or public experimental-tenant identifier; base, tokenizer and adapter hashes; training-data revision; target modules; rank and scaling; training precision; supported inference precisions; and held-out validation results. An adapter may be absent for a candidate, in which case the router selects the unadapted model or another eligible expert. The system must not silently substitute another customer's adapter. Public benchmark experiments can use declared repository or organization groupings as simulated tenants, but must not present those groupings as measured customer personalization.

A single base with several domain adapters is a useful alternative to several distinct small bases. It shares the expensive base weights and may simplify deployment, while allowing different task specializations. It does not guarantee the same breadth as a portfolio of independently pretrained models. The comparison should include one general adapter, domain-specific adapters on one base, and separate base-specific adapters, under the same training-data and total adaptation-budget accounting. Combining adapter outputs within one network is a different intervention from choosing between independently served language models; the two should not be pooled under a single "ensemble" label.

8.3 Reuse established multi-adapter serving

The first cloud implementation should use a maintained serving system's existing adapter support. Current vLLM documentation supports per-request LoRA selection for architectures implementing its `SupportsLoRA` interface and exposes limits including `max_loras`, `max_lora_rank` and `max_cpu_loras`. Compatibility is therefore checked for the exact checkpoint and engine revision; support for a model's unadapted inference does not imply support for all its adapter targets. The selected adapter and its resolved hash become part of every model event.[2]

S-LoRA and Punica explain why this architecture can be economical. S-LoRA manages adapter weights and KV memory through unified paging and schedules heterogeneous adapters; Punica batches computations that share a base while applying separate low-rank updates (Sheng et al. 2023; Lequn Chen et al. 2023). Their papers and implementation descriptions support reusing a base across

[1] Revision-pinned PEFT checkpoint format and hotswapping documentation, retrieved 25 October 2026. Source hashes and review scope are recorded in `adaptation-sources.json`.

[2] vLLM LoRA serving documentation. Per-request adapter support is explicitly conditional on the model interface and configured capacity limits.

tenants, not the claim that serving arbitrary different bases costs the same as serving one. Their reported throughput gains are historical experiments under their own workloads and baselines. They are not used as predicted gains for the proposed system, and an older README's statement about missing vLLM features is not treated as current: the inspected current vLLM documentation already describes LoRA serving.

For a desktop deployment, llama.cpp documents loading and scaling LoRA adapters and controlling adapters through its server interface. MLX-LM documents LoRA, QLoRA and full fine-tuning on supported models, as well as producing a fused model. These are implementation candidates, not evidence that the thesis's current checkpoint, export format and tool interface work together. The preflight must verify conversion, generation and held-out behavior after adapter fusion or quantization. Repeatedly merging and unmerging quantized weights is not treated as lossless; PEFT explicitly warns about accumulated rounding error.[3]

8.4 KV-cache continuity and the meaning of a switch

A key–value cache stores internal tensors computed from a token prefix under a particular model computation. It is not a portable transcript. Changing the base weights, architecture, tokenizer, position handling or relevant adapter generally invalidates reuse, even when the human-readable conversation is unchanged. A successful checkpoint load or a matching tensor shape does not establish cache validity. The safe default for a cross-base handoff is to re-tokenize the permitted state and recompute the recipient's prefix.

The distinction between three warm states is essential. A **warm base** has its weights resident; a **warm adapter** has its additional tensors available; a **warm conversation prefix** has reusable KV state for the exact computation. The first two can make loading cheap without making prefill free. Returning to a previous model may recover a retained compatible cache, but only if that state is still resident and its prefix identity matches. Changes to a browser screenshot or conversation ordering can invalidate the expected match.

Current vLLM prefix-cache documentation explicitly includes LoRA identifiers, multimodal-input hashes and tenant-isolation salts in its block identity. It also states that automatic prefix caching reuses matching prefixes to avoid repeated prompt computation; this does not eliminate the generation of new tokens. The design should preserve those correctness boundaries, not remove adapter identifiers merely to increase cache-hit counts. Customer authorization remains separate from cache optimization.[4]

A provider's cache-read billing category is another measurement. It may reward an eligible repeated prefix, but it is not proof that a particular tensor cache is available to the application or transferable to another service. API pricing, server-side cache observations and local cache residence are logged separately. Likewise, the browser's cookies, tabs and application state live in the controlled browser session, rather than in the language model's KV cache. A model handoff

[3] Pinned llama.cpp server documentation, MLX-LM fine-tuning documentation, and PEFT LoRA documentation.

[4] vLLM prefix-cache design and automatic prefix-caching feature description.

may preserve that external state while still losing computational reuse and some unrecorded reasoning context.

8.5 A useful exception: activated LoRA on one base

Activated LoRA (aLoRA) modifies when adaptation begins. Its weights activate from a declared invocation sequence; preceding tokens are processed with the base model's weights. The paper and current PEFT implementation explain how this preserves compatible base-model KV values before the activation boundary (Greenewald et al. 2025). This is directly relevant to using several specialized checks or continuations over an already processed context.

The guarantee is restricted. It applies to the shared base computation before activation, not to arbitrary different LLMs. KV values generated after an adapter has activated are not automatically reusable by the base or a different adapter; the reusable prefix must be identified, and contaminated suffix state discarded or separately retained. PEFT's documentation restricts aLoRA to causal-language-model tasks, requires an invocation sequence, and does not support merging these selectively applied weights into the base. Consequently, aLoRA is a promising secondary experiment for **one base plus several specialists**, not a reason to assume costless transitions throughout a heterogeneous portfolio.[5]

8.6 Select once, then switch at justified boundaries

The primary portfolio policy selects a model configuration for a whole task attempt. This minimizes handoff ambiguity and makes comparison with fixed-model baselines interpretable. A secondary policy permits changes at declared phase boundaries: for example, after repository investigation and before implementation, or after locating a browser form and before a specialized extraction step. Per-call switching is an ablation whose benefit must overcome its extra overhead; it is not the default merely because a routing decision can be computed cheaply.

Let $Q_j(s)$ predict terminal success from the current permitted state s when model configuration j continues. Let $C_j(s)$ and $T_j(s)$ predict remaining cost and time, excluding the transition. If k is the current configuration, switch to j only when

$$\begin{aligned}\widehat{Q}_j(s) - \widehat{Q}_k(s) > \lambda\big[\widehat{C}_j(s) - \widehat{C}_k(s) + \widehat{C}_{k\to j}(s)\big] \\ + \mu\big[\widehat{T}_j(s) - \widehat{T}_k(s) + \widehat{T}_{k\to j}(s)\big] + h.\end{aligned}$$

Here λ and μ convert monetary cost and latency into the declared utility scale, and $h \geq 0$ is a calibrated hysteresis margin against repeated reversals. Transition terms include loading, re-prefill, serialization, summarization, network transfer and queue delay as applicable. The quality predictor must be trained on the actual handoff state, including any summary loss; a fresh-start benchmark score is not a continuation-success probability. Already incurred expenditure remains in total task cost but is a sunk cost for this forward-looking decision. A remaining-budget constraint independently excludes unaffordable actions.

[5] PEFT activated-LoRA documentation and training example; the source register distinguishes inspected documentation from the paper's targeted full-text review.

The handoff record contains the goal, completed work, current repository patch or browser state reference, observed test or tool outcomes, unresolved questions, permissions and remaining limits. Tools must finish or be cancelled at a documented boundary. The model receives externally recorded work and observations, not an assumed transfer of opaque reasoning state. This makes semantic state loss observable and keeps model switching separate from changing tool privileges.

8.7 Desktop fine-tuning versus cloud adapters

Training location and inference location are independent design choices. A customer can run a model locally after its permitted adaptation was trained on a cloud GPU, or train on their own supported desktop. Local training is useful when data cannot leave the device, but full fine-tuning should be conditional on measured memory, runtime, power and thermal limits. A model fitting in quantized inference memory does not establish that full training fits.

For scale, a conventional mixed-precision AdamW configuration can require approximately 16 bytes per trainable parameter for weights, gradients, master weights and optimizer moments before activations and temporary buffers. A 4B-parameter model would therefore require approximately 64 GB for those quantities alone in that illustrative configuration. Optimizers, sharding, precision and offloading change this accounting; it is not a universal lower bound. LoRA and QLoRA reduce trainable-state requirements but still consume base-model, activation and runtime memory (Dettmers et al. 2023). MLX-LM's documented full-fine-tuning option establishes software functionality, not suitability for every desktop.

The initial product configuration should compare one locally resident compact model with a small remote specialist pool. Additional local bases are admitted only when their measured marginal benefit justifies memory, disk, loading and maintenance costs. In a shared cloud deployment, additional adapters on a common base may be cheaper than additional independently resident bases, but traffic mix and contention can reverse that advantage. Pool size and placement are therefore experimental variables rather than fixed beliefs about what "small" should mean.

8.8 Measurements and adaptation lifecycle

The switching study records cold and warm base loading, adapter loading, time to first token, prefix tokens recomputed, cache-hit observations, peak memory, queue time, end-to-end latency and final success. It compares task-level selection, phase-level selection and per-call selection under the same resource ceilings. Within-base ordinary LoRA and aLoRA are separate configurations. Cold-start and steady-state results are both reported; the experiment must not warm every expert for one policy while charging startup to its comparator.

Adaptation evaluation separates model improvement from allocation changes: first test the adapted experts under a frozen router, then recalibrate routing on development data and test that distinct policy. Training, teacher generation and repeated compatibility checks enter an explicit amortization ledger. A new model

release triggers an admission evaluation and, if selected, revised base-specific adapters or retraining; it does not inherit the old adapter's quality label. The serving system can keep the same customer-facing identity while changing its validated internal mapping. These mechanisms make a changing portfolio feasible without assuming that every new model improves the frontier or that every customer benefits from training an adapter for every candidate.

Chapter 9

Cost-aware portfolios across task domains

9.1 Expanded research object and relation to the two-model study

The research object is a **portfolio of eligible model deployments**, selected according to expected task success, total task cost and operational constraints. The portfolio may contain several small models, several large models and domain specialists. A small model can run remotely; a larger model can run on a capable local workstation. Parameter count, weight openness, deployment location and customer adaptation are separate attributes. “Free models” means an eligible open-weight pool without a mandatory per-call model-license charge where the applicable license permits that use; it does not mean zero inference, hardware, training or operational cost.

This chapter controls the expanded prospective protocol. The earlier local/remote pair is a restricted case with two candidates and remains a useful integration pilot and ablation. Its pairwise primary comparison is replaced by the portfolio comparison defined below. **Software-engineering issue resolution remains the primary confirmatory domain.** Browser interaction is a prespecified external replication with its own task population, harness and success criterion. Frontend development, backend development, languages and issue types are analysis strata or separately defined workloads, not interchangeable leaderboard scores. No pooled browser/SWE success rate will serve as the primary endpoint.

The aim is to approach the attainable accuracy–cost frontier of a declared pool on a declared workload. It is not a claim to attain the capabilities of every open model, to identify an unseen task’s best solver perfectly, or to match the outcome-informed union of all models without paying acquisition and routing costs. Whether specialists provide complementary errors, and whether a router can exploit them, are empirical questions.

9.2 Related work and reusable infrastructure

RouterBench formulates multi-LLM routing in terms of monetary cost and performance and releases precomputed outcomes for evaluating allocation

policies (Q. J. Hu et al. 2024). This supports reusable outcome matrices and comparisons across spending preferences. Its historical question-answering and coding-problem data do not measure repository repair, browser interaction or the costs of moving an active agent between deployments.

GraphRouter represents tasks, queries and candidate LLMs in a heterogeneous graph and predicts query–model relationships using performance and cost information (Feng, Shen, and You 2025). Its new-model experiment uses auxiliary few-shot interaction data and descriptions; "without retraining" must not be read as "without obtaining evidence about the new model." This is a relevant alternative to rebuilding a closed-set classifier whenever a candidate is admitted. It does not establish reliable cold-start routing for a newly released coding or browser model in this study.

BEST-Route jointly considers model choice and test-time sampling effort (Ding et al. 2025). It motivates treating model size and inference budget as separate controls: several attempts by a smaller model may compete with one attempt by a larger model. The primary portfolio experiment fixes one agent attempt per condition. Best-of-many generation, voting and judge-based selection are separate treatments whose generation, selection and verification expenses must all be counted. They cannot inherit single-attempt cost labels.

The current LLMRouter paper and official implementation provide a unified formulation and library for single-turn, multi-turn and personalized routers (Feng et al. 2026). Its documented components include context and model encoders, scoring, decision rules, supervision generation and multiple router families. The implementation is a preferred integration candidate alongside existing TTGL data-validation utilities, rather than a reason to build another general router framework. Adoption requires a pinned revision, license/dependency review and checks that its data interface preserves whole-agent outcomes, task splits and full cost records. This source review inspected the four papers' relevant formulation/method sections and the official repositories; it does not replicate their reported improvements. Retrieval hashes and revisions are recorded in `ensemble-methods-sources.json`.

9.3 Candidate identity, objectives and price sensitivity

At release epoch e, let $\mathcal{M}_e = \{m_1, \dots, m_K\}$ be the frozen candidate register. A candidate identifies a base checkpoint or service revision, adapter revision, tokenizer and prompt protocol, precision, inference configuration, deployment and tool interface. Thus the same base model on a laptop and on a rented GPU can define different treatments; they may have different latency, memory limits and total cost. Multiple adapters are distinct candidates only when their data provenance and target use are declared.

For observable state s and customer configuration u, the router predicts $\hat{p}_m(s,u)$, the probability of final task success under the specified continuation, and $\hat{c}_m(s,u)$ and $\hat{\ell}_m(s,u)$, its remaining cost and latency. Eligibility $\mathcal{E}(s,u) \subseteq \mathcal{M}_e$ enforces required modalities, tools, privacy permissions, context capacity, hardware availability and resource limits before ranking. Model size is a candidate attribute, not a rule that a larger model always has higher success probability.

A decision score is

$$V_m = \hat{p}_m(s,u) - \lambda[\hat{c}_m(s,u) + S^C_{a,m}(s)] - \mu[\hat{\ell}_m(s,u) + S^T_{a,m}(s)],$$

where a is the currently active deployment and S^C and S^T represent switching cost and delay not already included in the continuation estimates. The router selects the eligible action with highest score, subject to execution caps; a registered abstention or failure disposition applies when no action is eligible. Cost terms must be mutually exclusive so a repeated prefill is not billed twice. Any quality loss from summarization or changed context belongs in the continuation success model, not in an invented universal switching discount.

Here λ measures the value assigned to a dollar relative to success probability, and μ does the same for latency. A customer-facing price preference can map to these calibrated operating points or to a mean spending target B and minimum quality constraint. Predicting mean cost does not enforce a hard individual-task ceiling: the executor separately limits expenditure and records budget-exhausted tasks. The primary economic comparison uses dollar cost; latency is reported separately with a fixed service cap. A joint utility score is a secondary view with its weights disclosed.

The budget frontier is generated from a short grid frozen using calibration data. Increasing λ is intended to favor cheaper actions, but actual savings and quality must be measured. A router need not be an LLM: regularized success/cost predictors over frozen embeddings and metadata are the initial baseline, followed by one existing multi-model method if its incremental benefit exceeds inference and maintenance costs.

9.4 Task specialization and portfolio size

Permitted predictors include issue text, repository metadata, languages present before editing, frontend/backend indicators derived from declared files or manifests, task category, tool and observation requirements, context length and deployment state. A browser task may require DOM manipulation, screenshot grounding or both. Frontend code generation is not labelled browser navigation merely because a browser is used for testing. Labels derived from the final reference patch, hidden test failures or eventual success cannot be inference-time features.

The primary expanded pool contains eligible open-weight configurations; closed APIs remain external reference conditions or a separately labelled extended-pool ablation. Paid hosting of an open-weight model does not change that classification. The initial expanded pool has **three compatible base-model candidates per evaluated domain**. A useful design includes an inexpensive generalist, a higher-capability generalist and a plausible domain specialist, but admission is based on development evidence rather than those marketing labels. The same candidate may appear in more than one domain. Report both the number of distinct base checkpoints and the number of deployment/adapter configurations, so three variants of one base are not presented as three independent model families. The model catalog supplies a screening list; its published cross-harness scores do not determine an experimental winner.

The registered capacity ablations are $K = 2, 3, 5$, with an upper screening limit of six candidates per domain. Nested subsets are chosen on development data under

a fixed incremental-value rule. Selection considers residual successes, total cost, runtime compatibility and provisioning overhead, not just average benchmark rank. Global development domination is not enough to delete a specialist if it contributes on a predeclared task stratum; conversely, a family name or architecture difference does not prove useful diversity. Test outcomes cannot choose which pool size or members to report as primary.

One and multiple small-model deployments are explicit ablations. Report the number of base models that must remain resident, memory and storage footprints, load time, adapter residency, idle GPU allocation and expected utilization. A portfolio may improve success while worsening economic efficiency when several rented servers sit idle. Local single-model, cloud single-model and cloud multi-model policies therefore have separate resource accounts. Deployment is selected for the required workload and device profile, not inferred from a parameter-count threshold alone.

9.5 Whole-task routing and bounded handoffs

Whole-task routing is the primary intervention: choose one deployment before model execution and keep it for the bounded agent attempt. This preserves state continuity and makes paired model outcomes interpretable. It is different from choosing a model for every API request inside an agent loop. A task can contain many such calls, tool executions and retries while still using one deployment.

Secondary policies may switch at declared semantic checkpoints, such as after repository inspection, at a coding-to-browser verification boundary, or after a fixed failed-repair budget. The policy records why it switched and what state was transferred. A minimum dwell period or calibrated hysteresis can suppress repeated switching when estimated gains are small. The comparison includes always-sticky, checkpoint-switching and request-switching policies on the same tasks, with all handoff overhead included.

A KV cache contains model-specific internal representations. Sharing text or a browser/workspace checkpoint is not equivalent to sharing that cache. Re-tokenization, prompt reconstruction, prefill, lost provider-cache discounts and reloading weights or adapters can alter both cost and latency. Returning to a model may recover a retained cache only if the runtime preserves it and the prefix and configuration still match. The experiment records actual cache hits and misses instead of assuming every switch destroys all state or that incompatible deployments can reuse it. Handoff quality requires fresh continuation experiments; stitching together independent complete runs does not estimate a sequential policy.

9.6 Data acquisition, calibration and comparator selection

The development data are divided into router-training, calibration and model-selection partitions with grouping by repository or browser site/template family. Where independent clusters are insufficient, grouped nested cross-validation is allowed as a documented development substitute; an untouched final test is

still required. Training outcomes fit predictors, calibration estimates confidence and operating points, and the model-selection partition chooses the candidate pool, comparator and primary policy under the registered procedure. Customer adaptation examples are also excluded from final evaluation and from any supposedly unseen customer group.

On a complete-label development subset, every eligible candidate runs on the same tasks with independent clean states. This observes complementarity, including small-model successes where larger models fail. In incomplete operational logs, only selected-model outcomes may be observed. Those logs cannot label unchosen models as failures. Any later contextual-bandit treatment must log selection propensities, disclose exploration cost and evaluate assumptions needed for off-policy estimates. The initial study uses supervised complete-label acquisition rather than relying on an unvalidated online exploration policy.

The primary static reference $S^\star$ is the candidate with highest mean software-engineering success on the **model-selection partition**, among deployments meeting the frozen eligibility and per-task execution constraints. Ties are broken by lower mean total cost and then a deterministic identifier ordering. This comparator is not selected from final-test scores. The primary portfolio P and its single price-sensitive operating point are also frozen before testing. Selecting a winner on development data can still be noisy, so final tables report every evaluated static candidate and make no claim that $S^\star$ is the unknowable population-best model.

9.7 Comparisons and confirmatory analysis

The central comparison is P against $S^\star$ on software-engineering tasks. Additional baselines are the cheapest eligible fixed model, each other selected static model, a task-independent random mixture, a domain/stratum lookup rule, and the best registered two-model router selected on development data. The random mixture optimizes its probabilities on development quality/cost estimates at each registered budget; it is a stronger comparator than an arbitrary uniform mixture and corresponds to the attainable mixture of static points discussed in RouterBench. The random mixture and lookup rule are calibrated to estimated cost using development records; equal model-call share is not equal expense. Their realized costs are reported, including mismatches. Across-domain reporting additionally includes one fixed global model eligible in both domains, if such a configuration qualifies, and a per-domain fixed choice, to test whether coarse specialization already explains the benefit.

For the primary domain, the existing five-percentage-point quality tolerance is a proposed engineering margin to accept before freezing, not a statistical finding. Let $A_P, A_{S^\star}$ be mean success probabilities and $C_P, C_{S^\star}$ mean total task costs. The confirmatory sequence tests $A_P - A_{S^\star} > -0.05$ using a lower one-sided 95% confidence bound and, only after that gate passes, $C_P/C_{S^\star} < 1$ using an upper one-sided 95% bound. Estimates and intervals are reported regardless of gate passage. A smaller accepted tolerance requires revisiting the sample-size plan before test acquisition.

The planned full study retains three attempts per task/condition, averages within task, and treats repositories as dependence clusters in its primary bootstrap; uncertainty with few clusters receives the existing sensitivity analysis. Browser

replication uses site/template grouping and reports its own intervals. Pool sizes, language slices, browser modalities, customer adaptations, alternative budgets and switching policies are prespecified exploratory comparisons unless separately registered and funded. Neither a favorable subgroup nor a visually attractive frontier replaces the primary operating point after outcomes are known.

9.8 What it means to approach the combined frontier

Report dollars per assigned task and dollars per successful task, with the latter equal to **all assigned-run costs divided by total successes** under consistent repeated-attempt aggregation. Include failure expenditure, router inference, tools, retry policy and deployment allocation. Zero successes leave cost per success undefined. Report predictive calibration for each model, allocation regret, model-selection frequencies, latency, domain-specific success and cost distributions; do not interpret aggregate success as sufficient evidence of calibrated model choice.

An outcome-informed selector over all observed model outcomes is a diagnostic opportunity bound. The union event “at least one model succeeded” is an even more optimistic coverage summary. It may select sampling luck and assumes knowledge unavailable before execution. Running all candidates and selecting a successful answer would incur all their costs plus a fallible verifier. Therefore a union curve is labelled an oracle bound, not the ensemble’s deployable frontier or a promise to match all models for one model’s cost.

Complete-label development or diagnostic matrices can support offline policy analysis for whole-task selection. They do not measure online routing overhead, deployment-specific queueing or warm-cache behavior. The primary deployed policy and its static comparator require actual paired executions; diagnostic replay and live measurements are reported separately. Research acquisition spending for all candidate evaluations is also separate from the estimated cost of serving one future routed task.

9.9 Refreshing models and adapting customers

“Best at a given time” is implemented through versioned release epochs. A new candidate first passes interface and resource checks, then runs on a newly designated development panel. Its profile, cost model and routing calibration may be updated; the agent harness and all model weights need not be retrained merely because the candidate register changes. Compare no-update, profile-only update, router recalibration and full router refit as separate update strategies. New student adapters or full fine-tunes are additional interventions with their own training cost and independent quality evaluation.

Each epoch freezes its pool, rates/accounting scenario, policy and test manifest. A newly admitted model is evaluated on a fresh held-out panel or a prospective time slice; repeatedly optimizing against the same final benchmark converts it into development data. Apparent gains across epochs may come from model improvements, changed tasks, changed prices or router updates. An overlapping control set and unchanged baseline help distinguish these causes but do not restore independence to repeatedly inspected tasks.

Customer-specific routing and customer-specific model training are likewise

distinct. Evaluate a base portfolio and an adapted portfolio each with a frozen and recalibrated router. Test unseen tasks within customer and unseen customers separately. A shared dataset or adaptation procedure does not imply that one LoRA weight file is compatible with unrelated bases. The serving design identifies the base-and-adapter pair and accounts for each adapter's training, loading and retirement rather than treating customization as free.

9.10 Staged feasibility and execution boundary

The existing US$100 single-80-GB Runpod allocation remains an integration resource. Its two-model, 12-development-task pilot is retained if the expanded pool cannot fit; that would validate software and accounting only, not the multi-model hypothesis. A three-candidate pilot proceeds only after a costed compatibility run establishes that all assigned blocks, artifact export and cleanup fit the cap. Browser environments and any separately billed API use require explicit resource accounting and the existing authorization boundaries.

The expanded confirmatory matrix is priced after measuring model loading, task duration, failure rates and total cost on excluded development tasks. Capacity ablations and browser replication are staged after the primary pipeline works; no commitment to the full Cartesian product follows from listing its factors. Until funded runs and their registered analysis exist, this chapter specifies the experiment and the claims it could support, without asserting that the portfolio outperforms a fixed model or reaches the proposed frontier.

Chapter 10

Software-engineering evaluation and the restricted two-model case

This chapter supplies the repository-level benchmark procedures for the expanded portfolio study and retains the earlier two-model setup as an integration pilot and comparator. The ensemble methodology is the controlling protocol for the main claim: a frozen multi-model portfolio versus a strong fixed configuration chosen on development data. The two-model case below is not a competing confirmatory primary. No repository-level results have been collected, and the scope change precedes those outcomes.

The full study addresses the six research questions in the introduction. On the primary software-engineering domain, the proposed quality gate is no more than five absolute percentage points below the declared fixed reference, followed by superiority in the ratio of mean total costs. That margin is an engineering tolerance to be accepted before the final freeze, not a measured property or a universal customer preference. The ensemble protocol specifies the selected configurations, operating point, statistical gate and multiplicity boundary. Additional domains and policy variants do not silently add confirmatory hypotheses.

10.1 What “single LLM” means in this comparison

A model without tools and a coding agent with repository access do not receive equivalent opportunities to solve a software issue. The primary single-model baselines therefore use the same agent scaffold as the portfolio system. The difference is whether every model request is served by one fixed backend or whether the registered policy can choose between backends. Repository access, shell execution, test commands, edit mechanisms, initial instructions and termination rules remain identical within a comparison.

A bare-LLM condition may be added as a secondary diagnostic. It receives the issue and a fixed, automatically selected repository-context package and returns a patch without interactive tools. It is explicitly labelled a different interface condition. Its contrast with an interactive agent estimates the combined effect of feedback, tool access and repeated inference; it cannot isolate routing or establish that two

models are intrinsically superior to one. The context selector is frozen, does not consult the reference patch, and is shared across bare-model conditions.

The study also separates pre-call routing from escalation. In pre-call routing, the router selects one backend for an entire agent attempt before that backend generates an answer. In escalation, a local agent performs bounded work before a remote agent continues from its state. Escalation incurs the local attempt's cost and creates a different context for the remote model. An archived remote-only trajectory is not a counterfactual for that continuation. Pre-call routing is the initial primary treatment because it admits cleaner paired measurements; a real escalation treatment requires fresh executions and is a secondary experiment.

10.2 Benchmark selection and target population

The primary benchmark is SWE-bench Verified, the documented subset of 500 repository issues whose solvability was examined by software engineers (Jimenez et al. 2024; SWE-bench Team 2024). A task comprises an issue statement, a repository and base revision, and an evaluation procedure for the produced patch. The primary outcome is the benchmark evaluator's resolution decision. This measures success against the specified issue tests and regression checks, not general maintainability, security, review acceptance or production fitness.

The selected SWE-bench dataset revision, task-repository revision, evaluator revision and container image digests are fixed together. The current repository documents a version-five command interface and a separate task repository; historical command examples must not be mixed with that interface without checking compatibility. The evaluator's reference-patch check is used on a small preflight subset to establish that images and tests work. Reference patches, hidden test additions and evaluator internals remain outside the agent's accessible workspace. The preflight checks validate the environment, not the candidate model.

The full Verified set offers an interpretable common benchmark, but it is old enough to be vulnerable to pretraining exposure. "Verified" establishes neither model novelty nor absence of contamination. Model-card training-cutoff claims, release dates, public benchmark exposure and any retrieved issue/patch overlaps are documented. No automatic decontamination procedure can prove that an opaque provider never trained on a task. Accordingly, the thesis distinguishes controlled within-benchmark comparisons from claims about unseen customer repositories.

A second benchmark is reserved for external replication after the main method is frozen. SWE-bench Pro is a suitable candidate because its authors provide repository-level, longer-horizon tasks. Its current public version-two documentation specifies 642 tasks across 11 repositories in Harbor format and separates that version from the legacy version-one pipeline (Deng et al. 2025; Scale AI 2026). Any use of this corpus must fix the version, image digests and locked evaluation protocol. It must not pool version-one scores with version-two outcomes or assume the public split duplicates a private leaderboard. External replication will proceed only after repository/task overlap and access requirements are audited; it is not part of the currently funded feasibility pilot.

LiveCodeBench or comparable coding-problem datasets may be used to debug

feature extraction, model endpoints and training mechanics. They are not the primary evidence for repository-level software engineering. Existing TTGL coding-problem artifacts are retained as engineering history and are not inserted as the missing results of this prospective study. Examples or outcomes already examined while developing the method are classified as development material.

10.3 Development data, test isolation and repository familiarity

Router development requires outcome and cost observations for the eligible candidates on shared tasks; the restricted two-model case observes both backends, and the main portfolio requires the acquisition design in the ensemble protocol. These paired runs are collected on development issues that are excluded from the final Verified evaluation. Candidate development data may come from non-Verified SWE-bench tasks or a separately licensed issue corpus. Subtracting Verified identifiers alone is insufficient: the preparation pipeline must also remove duplicated issue text, related pull requests, shared reference patches, and clearly equivalent bug instances. Exclusion rules are documented and applied before model comparison. Ambiguous clusters are kept together or excluded from development.

The study reports two distinct generalization estimands. The primary full-Verified comparison is issue-disjoint: none of its task outcomes or solutions may train, calibrate or select the router. Some repositories can nevertheless be familiar through different development issues. A second, predefined repository-unseen subset of Verified is selected using repository identity and task metadata before any candidate outcomes are inspected. All development issues and adaptation examples from those repositories are excluded. Results for that subset estimate a narrower form of cross-repository transfer. Scores from a subset are not labelled as full-500 leaderboard results.

The repository allocation is established in a signed or hash-recorded manifest. Among remaining eligible development repositories, the expanded study separates router-training, calibration and model-selection partitions as specified in the ensemble protocol, grouping by repository when feasible. If independent clusters are insufficient, grouped nested development cross-validation is the documented substitute. A within-repository chronological split may support the restricted integration pilot only and is explicitly the weaker design; it does not remove the expanded study's model-selection partition or untouched final test. The task manifest includes repository, issue identifier, issue timestamp, base commit, related-issue cluster, split and exclusion reason. Test task IDs may be known to orchestration and the evaluator but are not predictor features. The router must not receive the official resolution label, a reference patch, hidden-test feedback or a benchmark-specific difficulty label at decision time.

After the primary test is opened, no feature, model, calibration method, threshold or utility weight is revised on the basis of its outcomes. Any subsequent change defines a new exploratory version and requires a new untouched test set for confirmatory claims. This includes changes motivated by a visually attractive frontier or by a subgroup failure. A time-stamped internal freeze is useful, but it must not be described as external preregistration unless the protocol is actually deposited in an independent registry before execution.

10.4 Common agent scaffold and execution controls

The preferred research scaffold is mini-SWE-agent, whose current documentation describes a small agent with shell-based interaction, linear conversation history and support for multiple model providers (SWE-agent Team 2026). The SWE-agent maintainers currently recommend mini-SWE-agent as the simpler successor for new use (Yang et al. 2024). Reusing this scaffold reduces implementation burden and helps make model substitution interpretable. Its published performance claims are not evidence for the proposed TTGL routing intervention.

The scaffold, prompt template, action parser, model transport, container environment and stopping policy are pinned. A compatibility suite checks each candidate model before inclusion: valid action formatting, tool-response continuation, context accounting, maximum-output handling, cancellation, malformed-response recovery and preservation of task identity. If a provider requires a different transport wrapper, that wrapper may translate protocol fields but must not add a stronger planning procedure or extra hidden context. Native reasoning and tool-calling differences are recorded as treatment characteristics.

Each task/condition/seed begins from an independent clean checkout and an identical prebuilt environment. Reuse is allowed for immutable container layers and dependency caches; writable repositories, conversation state and credentials are not shared between runs. Task order is randomized or counterbalanced across models and policies to reduce confounding from time-of-day load, provider degradation or cache warming. Concurrency limits are fixed and recorded. Model inference runs separately from the sandbox that executes repository code.

Two resource regimes are reported. The common-resource regime applies the same wall-time, agent-step and spending ceilings to all conditions, with limits selected on development tasks and frozen before test. It estimates performance under an explicit application budget. The service-profile regime uses each provider's documented recommended inference mode within a disclosed economic budget, because identical numerical token caps do not imply identical reasoning effort or cost across tokenizers. These regimes answer different questions and are not pooled into one ranking.

No model receives hidden evaluator results during its attempt. Ordinary repository tests available at the base revision may be run by every agent, and their execution cost is recorded. Final grading occurs in a fresh evaluation environment. The pinned SWE-bench documentation warns that cached evaluation results can depend on run identifier and instance identifier rather than patch content; therefore each model, policy, seed and patch receives a unique run identifier, and the evaluator record is checked against the submitted patch hash. Reusing a previous success after changing a patch is treated as a failed run-validation check.

10.5 Model eligibility and restricted-pair experimental arms

Model selection follows a dated evidence review, not an unqualified claim to have tested "the latest model." A model is eligible when an exact downloadable checkpoint or stable API identifier is available, its intended use and license are understood, it supports the frozen scaffold, and it can complete development

compatibility checks within the available resources. A rolling provider alias is recorded together with the resolved snapshot or response metadata; if the provider cannot expose a stable revision, that limitation is visible in the result table. New releases after the freeze are considered for a separate replication rather than silently replacing a running condition.

The restricted-pair shortlist uses **Qwen3.5-4B** as a small-model candidate and **Qwen3.5-9B** as a capacity/hardware ablation, subject to the model-card and compatibility checks recorded in the accompanying model register. **DeepSeek-V4.1-Flash-1017** is the proposed remote reference when an endpoint can be tied to that exact version; **Qwen3.8-27B** is an alternative open-weight remote control whose serving requirements must be measured. A provider alias is not assumed equivalent to the named checkpoint. These choices supersede legacy Qwen2.5 development configurations for the prospective study; the existing trainer and serving stack have not thereby been shown compatible with the newer architectures.

An advertised low active-parameter count for a mixture-of-experts model does not make its total weight footprint small. In particular, a single 80 GB training GPU is not a deployment plan for the full DeepSeek teacher. Closed-model reference systems include at least one contemporary OpenAI model and one contemporary Anthropic or Google model where authorized access and budget permit. Exact identifiers, quantization, context limits and provider prices are resolved in the model register before execution. The study does not assume that every shortlisted condition can be funded by the initial pilot budget.

Arm	Model allocation	Purpose and status
L	Local candidate for the complete agent attempt	Measures local capability and operating cost
R	One remote open-weight candidate for the complete attempt	Restricted-pair reference; stronger performance is tested rather than assumed
D	Learned pre-call local/remote selection	Restricted two-model baseline
RAND	Random pre-call selection with frozen calibrated allocation probability	Allocation baseline, evaluated with the same local/remote pair
HEUR	Prespecified prompt-length or development-fitted simple rule	Tests whether learned routing exceeds a simple allocation rule
UNC	Router ranking predicted local failure	Separates uncertainty from predicted benefit of delegation
ESC	Bounded local work followed by registered escalation	Secondary real cascade; requires fresh continuation runs
C1/C2	Each selected closed model alone in the same scaffold	External economic/accuracy reference conditions
ORACLE	Outcome-informed selection using paired completed runs	Nondeployable diagnostic bound only
BARE	Single model with frozen context and no interactive tools	Separate interface diagnostic; not the primary routing comparator

Only L, R, D and RAND are necessary for the first software-engineering feasibility pilot. D versus R characterizes this restricted baseline after its pair and policy

are frozen. The main confirmatory comparison instead uses the multi-model portfolio and fixed reference declared in the ensemble protocol. Additional candidates increase acquisition cost and must meet that protocol's development-only admission rules. A cheap-to-query open-weight model served by an external provider is not labelled local execution; deployment location and model openness are separate columns.

10.6 Two-model router baseline and deployment policy

For task context x, let $p_L(x)$ and $p_R(x)$ estimate local and remote resolution probabilities, and let $c_L(x)$ and $c_R(x)$ estimate the expected costs of completing an attempt. The basic benefit score is $g(x) = p_R(x) - p_L(x)$. A cost-aware score compares this estimated gain with incremental cost, for example $s(x) = g(x) - \lambda[c_R(x) - c_L(x)]$. The multiplier λ expresses a declared willingness to pay for expected additional resolution probability. A ratio formulation is possible but requires care near zero or negative incremental costs; this baseline uses the difference formulation.

Initial predictors use transparent prompt-derived features and a regularized linear classifier, followed by a fixed pretrained embedding model as a separate candidate. Vocabulary, feature scaling, cost models, calibration and any router-model selection use development data only. A small classifier is preferable to an additional generative model unless the latter's benefit survives inclusion of its own inference cost. The chosen feature set and classifier are frozen before test. The meaning of its confidence is task-resolution probability under a particular backend and scaffold, not the model's natural-language declaration of certainty.

Paired development outcomes permit estimating benefit without assuming that the remote model always dominates. Each backend is run independently on the same development tasks and repeated seeds. A task with local success and remote failure is retained. A router trained only on failed local tasks would not observe the full deployment distribution and could overestimate escalation value; historical production traces are therefore insufficient unless their sampling policy is known and the resulting selection bias is addressed.

The two-model online baseline uses a threshold selected on calibration data; the multi-model policy generalizes this to calibrated configuration utilities. It cannot choose the best half of an already observed test batch, because that assumes knowledge of future arrivals. Candidate thresholds are fixed on a declared grid, and one is selected using a prespecified calibration utility or quality constraint. The realized test exposure and cost are measured; they need not equal the calibration target. A fixed-budget batch-ranking analysis may be reported separately for queued workloads and explicitly identified as such.

RAND uses a frozen probability derived from the same calibration target and uses a recorded random seed. Because tasks consume unequal token counts and time, matching the number of remote calls does not ensure matching actual cost. Comparisons therefore report both remote exposure and measured cost. A supplementary matched-estimated-cost allocation experiment can use the same development-fitted cost estimates for every policy, while a post-hoc interpolation of measured curves remains descriptive. No realized test-cost threshold is tuned to favour the learned router.

10.7 Escalation and confidence features acquired during execution

The ESC treatment allows local work for a registered maximum number of steps or cost fraction, then evaluates an escalation rule. Available features may include invalid actions, unsuccessful repository tests, repeated edits, context exhaustion and progress in the visible workspace. These signals cost time and inference to acquire; their cost is charged before assessing whether escalation was worthwhile. Feature extraction must not inspect the hidden grader, reference patch or final success label.

At escalation, the system transfers a canonical issue description, current workspace diff, executed commands, observations, unresolved plan and remaining budget. Switching occurs between completed tool actions. Opaque provider reasoning state or key/value caches are not assumed transferable. A summarizer, if used, is fixed across conditions and charged to the task. The remote model's performance on this altered state must be measured through actual continuation, because earlier local changes may help, mislead or damage the starting point.

Confidence-aware speculative decoding motivates the principle of allocating computation according to predicted useful work. It does not supply a task-level correctness guarantee. A draft-token acceptance head predicts acceptance within a particular verification algorithm; the routing model predicts issue resolution under a tool-using policy. Their targets, training labels and failure modes differ. Token log probabilities may be explored as additional signals when available, but they are not interchangeable with calibrated software-engineering success probabilities.

10.8 Outcomes and routing diagnostics

The primary software-engineering metric is resolved fraction: completed candidate patches that pass the pinned issue-resolution criteria divided by all eligible assigned task attempts. Syntax errors, invalid patches, exhausted budgets and model-originated failures remain failures in the intention-to-evaluate analysis. Infrastructure failures are separately labelled and may receive one prespecified retry with identical settings when a logged independent service failure occurs. The original failure, retry cost and final disposition remain recorded; model failures are not relabelled as infrastructure problems to improve scores.

Patch validity, regression-test status, time to final patch, number of tool actions, context/token use, and user-visible failure categories are secondary measures. A blinded human review of a prespecified sample can assess patch clarity, scope and maintainability, using a rubric and inter-rater agreement. Human review is not silently substituted for benchmark resolution. Where two models produce equivalent acceptable patches but differ in style, the primary binary outcome remains the evaluator's stated criterion.

Router quality is assessed at several levels. Predictive calibration uses Brier score, log loss and reliability diagrams on unseen tasks, with binning fixed before evaluation. Discrimination metrics such as AUROC and average precision are useful diagnostics for success or beneficial-delegation labels, but a high

AUROC alone does not establish a useful operating threshold. Decision quality is measured through resolved fraction at each registered cost point, regret relative to the outcome-informed oracle, and the rate of missed beneficial delegations and unnecessary expensive delegations. The oracle is computed after outcomes are collected and never used for model selection.

Coverage–risk curves describe error among tasks kept local as local coverage changes. Accuracy–cost curves describe the complete system, including delegated tasks and overhead. Both are needed: a router can make its local subset look accurate simply by delegating almost everything. A small set of predeclared cost weights or calibration operating points defines the prespecified descriptive frontier; dense post-hoc curves are labelled exploratory. Confidence bands or paired uncertainty estimates accompany comparisons where the sample size supports them.

Strata include repository, issue type if reproducibly annotated without outcomes, patch-size proxy available before inference, context length, local device profile and model version. True reference-patch size may be reported retrospectively as an audit characteristic but cannot be a deployment feature. Repository-level analyses are especially useful for identifying concentration of success or cost. Small strata are reported with denominators and uncertainty rather than promoted as independent discoveries.

10.9 Economic accounting and pricing comparisons

The principal economic metric is total cost per assigned task; cost per resolved task is reported alongside it as the total cost of all attempts divided by the number resolved. Failed attempts are included in the numerator. If no task is resolved, the latter quantity is undefined or unbounded and is not replaced by zero. A system that is cheap because it produces very little useful work must not be ranked solely by average API spend.

For a provider API, task cost sums the billable categories actually reported by that provider: uncached input, cached input reads, cache creation where charged, output and reasoning tokens where separately billed, tool or search fees, retries and any batch/priority modifiers. Tokens are counted with the model's accounting conventions. Output characters are not treated as tokens, and identical nominal token counts across different tokenizers do not guarantee equal information or compute. Price quotes are dated, currency-labelled and tied to the exact provider endpoint and service tier. Promotional credits and free tiers are reported separately from ordinary prices.

For local or rented inference, cost includes occupied accelerator/CPU time, storage and transfer charges, startup and model-loading time, and the overhead of the router and sandbox. A marginal-use scenario may treat already-owned hardware as sunk capital; a fully allocated scenario includes hardware amortization, idle capacity and electricity. Both are shown when relevant, rather than describing local inference as economically costless. Energy is measured only with a documented sensor or power source; an assumed wattage produces a scenario calculation, not an empirical energy result.

The restricted two-model pre-call treatment has cost $C_{\text{router}} + (1-a)C_L + aC_R + C_{\text{sandbox}}$, with a indicating remote selection. A local-first cascade instead includes

the local work even when it later delegates: $C_{\text{router}} + C_{L,\text{spent}} + aC_{R,\text{continuation}} + C_{\text{sandbox}}$. Verification and summarization are added where they occur. Training and model distribution are allocated separately over a stated usage horizon, enabling a break-even calculation without pretending that future task volume is known.

Price-only comparisons and accuracy comparisons are distinguished. Official list prices can be compared after normalizing billable units, but they do not establish cost per resolved issue. Published leaderboard accuracies belong to particular model, scaffold, dataset version, budget and evaluation settings. They are recorded as external evidence with those fields and are not combined with our measured prices to produce a supposedly controlled ranking when the conditions differ. The strongest comparison reruns eligible open-weight and closed-model systems under the same protocol; otherwise the thesis presents an explicitly qualified landscape table.

10.10 Statistical analysis and sample-size planning

All conditions run on the same eligible tasks, enabling paired analysis. Three independently recorded decoding seeds are planned for the adequately funded confirmatory experiment. Task-level resolution probabilities are estimated from those attempts, and the primary average weights tasks equally. Seeds are repeated observations within a task, not additional independent software issues. Provider nondeterminism may persist despite requested seeds; version and timing metadata are retained.

The primary uncertainty calculation uses a paired hierarchical bootstrap that resamples repository clusters and retains matched task/condition observations within sampled clusters, with task-level repeated-seed records kept together. Repository weighting and task weighting are fixed in the analysis specification so the reported quantity remains the task-weighted benchmark average. A task-stratified sensitivity analysis conditional on the observed repositories is also reported. The limited number of repositories means that even 500 issues may provide weak evidence for generalization to arbitrary new repositories; bootstrap intervals are not advertised as eliminating that limitation.

The main confirmatory noninferiority and cost claims follow the fixed gatekeeping sequence in the ensemble protocol, with the portfolio and its development-selected fixed reference as the contrasted conditions. A joint plot of paired accuracy differences and cost ratios helps interpret cases where quality and cost move in opposite directions. Secondary comparisons between learned routing, uncertainty routing, heuristics and extra models are descriptive with intervals; any formal family of significance tests uses a prespecified correction. Pareto dominance is assessed with uncertainty rather than inferred from overlapping point estimates alone.

The feasibility pilot estimates operational cost, variance and failure modes; it does not supply a final power claim. Before confirmation, sample-size simulation uses development-only estimates of portfolio/reference discordance, repository clustering and run-to-run variability. It estimates the probability of establishing the registered margin and cost effect across plausible scenarios. If the full benchmark remains underpowered, the remedy is to collect additional independent tasks or narrow the stated conclusion, not to relax the margin after seeing test results. A

failed noninferiority test is inconclusive about equivalence; a failure to detect a routing gain does not establish that routing has no value.

10.11 Student adaptation and logit-distillation extensions

The adaptation experiment starts only after a base-model reference and the unadapted router are fixed. Teacher trajectories come exclusively from development repositories and task clusters. They include observable issue context, visible commands, workspace changes and verified final patches. Hidden chain-of-thought is neither required nor presumed available. Teacher outputs are filtered through independent executable checks where feasible, and the filtering rate, failed examples and resulting task distribution are recorded.

The minimum comparison is unchanged student versus supervised fine-tuning with LoRA on verified teacher demonstrations. A token-matched same-family-teacher or additional-training control separates the source of demonstrations from the effect of extra optimization. Existing Transformers, PEFT and TRL components are preferred over a bespoke trainer. Exact weights, tokenizer, chat template, precision, rank, optimizer, learning schedule, data order and stopping rule are frozen, and three training seeds are planned for the funded adaptation study. A small pilot can check that loss and serialization work but cannot replace held-out issue evaluation.

Token-level distillation is a separate treatment. It uses a temperature-scaled KL term between teacher and student distributions only after verifying matching token identities, vocabulary coverage, sequence alignment and chat-template conventions. A shared model family is not sufficient evidence by itself. DeepSeek and Qwen tokenizers cannot be naively aligned by token index; verified-sequence distillation remains valid without that assumption. Top-k API log probabilities omit probability mass and are not labelled full-vocabulary teacher logits. If a truncated-distribution objective is used, it is defined and tested as a different approximation.

Preference learning is also distinct. DPO can consume chosen/rejected pairs labelled by executable checks, human review or an AI judge, but those label sources answer different questions. RLAIF specifically denotes AI feedback, whereas a unit-test reward is an executable verifier. A future RLAIF treatment must fix the judge, rubric, sampling scheme and reward-hacking checks. Online RL is not assumed to fit within the initial compute allocation. After any adaptation, the student is evaluated both with the frozen router and with a recalibrated router using new calibration data, separating model improvement from improved allocation. The final test remains unavailable to either training process.

10.12 Resource plan, execution stages and stopping rules

The current authorization is a single 80 GB Runpod GPU with a US$100 total Runpod ceiling for the initial training/inference experiment, including its storage, startup and shutdown costs. It does not authorize additional API-provider spending or fund an unspecified full matrix of contemporary models on 500 repository issues with three repeats, plus adaptation and external replication. External API usage

requires a separately costed and authorized allocation. Container execution and all ancillary infrastructure must also appear in the prospective resource plan. Access and live prices are prerequisites to allocating compute.

The first repository-level pilot uses 12 development issues sampled by a fixed seed from six eligible development repositories where possible, two issues per repository, before outcomes are observed. It runs L, R, D and RAND once per issue, at most 48 task trajectories. The router for this pilot is trained on separate development issues; if paired training labels are not yet available, a fixed rule is substituted and the pilot is explicitly an integration study, not a learned-router comparison. Pilot tasks never enter the final confirmatory test.

Pilot spending is allocated before launch: a reserve is held for storage, output retrieval and termination; live per-task and per-hour limits determine how many complete paired blocks can be run. The intended 48 trajectories are a maximum, not a promise that they fit the cap. A paired block is started only when enough budget remains for every required condition under its registered cap. If funds are insufficient, execution stops at the last complete block and the achieved sample is reported as a feasibility sample. No outcome determines which unfinished tasks are discarded.

The pilot first validates two development tasks and the grader, then performs the bounded paired run, then stages one student-training smoke run if funds remain. Only after measured throughput, accuracy variability and total cost are available is a priced proposal made for the confirmatory matrix. A process timeout is paired with a controller deadline and verified resource termination, because ending Python does not end GPU billing. Persistent volumes and interrupted jobs are accounted for. No unrelated resource is stopped or deleted.

10.13 Reproducibility, artifacts and completion criteria

The evidence-catalog cutoff of 25 October 2026 is separate from the execution freeze. Checkpoint and runtime revisions are recorded before any paid development run. The selected portfolio and fixed reference, router features and operating point, task splits, evaluator, harness, resource caps and model/service revision manifest are locked before the first held-out test call. Undocumented provider drift or a changed rolling alias triggers a separate versioned condition or a suspended run; outcomes from differing revisions are not silently pooled.

Each run records a task manifest, model/provider revision, agent and evaluator commits, container digests, prompts, routing policy and threshold, random seeds, raw tool/model events, submitted patch hash, usage counters, costs and grader identity. Outputs are append-only or written to unique run directories. The system refuses to reuse a cached success when the patch, environment or condition changes. Authentication values are never included in research artifacts.

The pre-results implementation specification requires four adapters around maintained components: a model endpoint adapter, a router decision hook, a cost/event recorder and a benchmark export/regrading adapter. Existing TTGL helpers can supply validation and analysis patterns, but the software-engineering runner must be tested independently. Required integration checks include exact task mapping, clean task resets, model-switch state preservation, tool-protocol errors, unavailable endpoints, budget exhaustion, cancelled runs, cache

invalidation and verification of output provenance.

A benchmark run is complete only when assigned tasks have final dispositions, grader outputs correspond to current patches, every failed or skipped attempt is accounted for, costs reconcile with provider or instance records, and analysis is reproducible from frozen inputs. A model-training run is complete only when its resulting artifact is saved and hashed; a claim of improved software-engineering performance additionally requires the independent held-out evaluation. The final thesis will contain measured results only after those conditions are satisfied. Until then, the contribution is the literature synthesis, explicit experimental design and reproducible implementation plan, without implied evidence that the portfolio already achieves the proposed accuracy–cost improvement.

Chapter 11

Analysis and reporting plan

11.1 Estimands and success criteria

The principal system estimand is the difference in issue-resolution probability between the frozen portfolio policy and the declared fixed-model reference selected on development data on the target task population. Its primary economic estimand is the ratio of mean total cost per assigned issue, portfolio divided by fixed reference, under a specified deployment/accounting scenario. The paired mean cost difference is a supporting estimand. A claim of useful savings requires both acceptable quality and lower cost. A system that is cheaper because it abandons difficult issues must not be described as equivalently capable.

The primary analysis follows the gatekeeping procedure specified in the ensemble methodology: first assess the declared noninferiority condition for resolution, then assess cost superiority if that gate is passed. The router-specific comparison with a baseline matched on estimated cost using calibration data addresses a different question—whether allocation is useful—and is a prespecified exploratory contrast. Actual realized cost is reported because calibration does not guarantee an exact spending match on test tasks. An exploratory configuration that happens to look best is not substituted for the primary system after inspecting the final test.

Let $Y_{ir}^{a} \in \{0, 1\}$ indicate resolution for task i, replicate r, under condition a, and let C_{ir}^{a} be its total recorded cost. For the three-repeat confirmatory design, define task means $\overline{Y}_{i}^{a} = \frac{1}{3}\sum_{r=1}^{3} Y_{ir}^{a}$ and $\overline{C}_{i}^{a} = \frac{1}{3}\sum_{r=1}^{3} C_{ir}^{a}$. The paired task differences are $D_{i}^{Y} = \overline{Y}_{i}^{\text{portfolio}} - \overline{Y}_{i}^{\text{fixed}}$ and $D_{i}^{C} = \overline{C}_{i}^{\text{portfolio}} - \overline{C}_{i}^{\text{fixed}}$. The primary cost ratio divides the across-task mean of $\overline{C}_{i}^{\text{portfolio}}$ by the corresponding fixed-reference mean; it does not average per-task ratios. Assignment to all eligible tasks is the analysis denominator. Infrastructure failures, timeouts and budget exhaustion are handled by the declared failure policy, with a sensitivity analysis that preserves the initial analysis rather than selectively deleting inconvenient cases.

Repeated seeds estimate stochastic variability within a task/condition. They are not independent repositories or independent task draws. The report states whether a metric estimates a single attempt, the expected success of one sampled trajectory, or the success of a budgeted multi-attempt system. Reporting pass@k as if it were single-attempt resolution would obscure both sampling effort and cost.

11.2 Accuracy and cost frontiers

For every prespecified policy configuration, the report plots mean issue resolution against mean total cost. Markers identify portfolio epoch and membership, routing policy, intervention point and deployment scenario. Uncertainty is attached to both coordinates or to pairwise contrasts as appropriate. Policies are considered dominated when another evaluated configuration is no less accurate and no more expensive under the same accounting assumptions, with uncertainty acknowledged. An empirical Pareto frontier is descriptive and does not establish optimality among untested policies.

Two views are required. A **fixed-quality view** asks how much cost can be reduced while remaining within the quality constraint. A **fixed-cost view** asks how much resolution improves at the same available spending. Remote-call fraction is also reported, but is not the horizontal axis for all economic conclusions: different requests can have different input/output lengths, cache behavior and reasoning budgets. A policy that sends fewer calls remotely can still cost more.

API and self-hosting scenarios are analyzed separately. The same open-weight model can be evaluated as a provider-hosted endpoint and as a self-hosted service, but the change in serving stack can alter latency, quantization and outputs. Prices from a dated catalog are not substituted for invoices when an actual call is made. Catalog-based extrapolations are labelled estimates, and intervals from task resampling do not imply certainty about future provider prices.

The primary inference concerns software engineering. Browser-task success, visual grounding and frontend acceptance are reported under their own evaluators and denominators. A global score requires workload weights fixed before outcomes; equal weighting is a chosen deployment scenario, not a discovery about the task population. Dollar-per-success ratios are computed within each domain before any declared workload aggregation.

11.3 Router quality as a prediction problem

A routing model should be assessed independently of the end-to-end system it controls. The study therefore reports predictive discrimination, calibration and decision quality.

Discrimination measures whether higher predicted success corresponds to higher observed success. AUROC and, where class imbalance makes it useful, precision–recall summaries can describe candidate-specific outcome predictors. A high score does not guarantee good allocation because the router needs to identify differences between models, not merely easy tasks. Benefit ranking is assessed through matched-budget resolution, opportunity regret and gains over a random policy.

The Brier score for a binary outcome is

$$\mathrm{BS} = \frac{1}{n}\sum_{i}(\hat{p}_i - Y_i)^2.$$

When multiple samples estimate a task’s success probability, the report specifies whether it scores individual Bernoulli outcomes or task means; these are different

quantities. Reliability plots show bin counts and uncertainty. Expected calibration error is reported with the binning rule fixed in advance, since changes in bins can materially alter it. Proper scoring rules are primary calibration summaries; an attractive reliability plot alone is insufficient.

For an action a_i, a simple observed utility is $U_i(a_i) = Y_i(a_i) - \lambda C_i(a_i)$. An outcome-informed oracle can define an upper bound by choosing the best observed action for each task. Regret to that oracle exposes headroom, but the bound is optimistic when it selects among noisy outcomes. It is diagnostic only, never an achievable deployed baseline or a way to choose test thresholds.

Selective-risk curves report error among tasks retained locally as a function of local coverage. Coverage must count actual local-only completion; a task using a remote planner or judge cannot be silently classified as local-only. The report also shows unresolved tasks, abstentions and remote escalations rather than counting only accepted local outputs.

11.4 Statistical uncertainty and dependence

Paired analysis uses the same tasks across conditions, reducing variation attributable to the task mix. Bootstrap or model-based uncertainty must respect the dependence structure specified by the dataset: trajectories within tasks, tasks within repositories, and potentially related issues or patches. The primary uncertainty scheme is fixed before observing test outcomes and is accompanied by the number of independent repositories/clusters.

If the available benchmark contains few repositories, a large number of tasks does not create equally many independent repository samples. Results can characterize the benchmark more precisely than they characterize arbitrary unseen repositories. Repository-level resampling, leave-one-repository-out sensitivity and the prespecified unseen-repository subset help reveal this distinction, but none makes a small set representative of all software development.

For ratios such as cost per resolved issue, uncertainty is computed from the paired numerator and denominator within each resample, not by independently combining marginal intervals. A denominator of zero produces an undefined value, which remains visible. Cost distributions are often skewed; the report includes median, upper quantiles and extreme-run attribution in addition to means.

Subgroups, multiple budgets and model portfolios create multiplicity. The methodology identifies the confirmatory contrasts and correction/gatekeeping rule. Other comparisons are explicitly exploratory. The thesis will report negative and inconclusive outcomes and will avoid treating a failure to reject a null as proof of equivalence.

11.5 Power and the role of the feasibility pilot

A limited pilot serves to validate tooling, task duration, hardware memory, logging and price estimates. It is not described as a well-powered test of a small accuracy difference. The number of pilot tasks and stopping rule are frozen before outcomes are seen; a pilot task used for tuning is ineligible for a later untouched test.

A simple planning approximation illustrates the scale required for paired binary comparisons. If q is the probability that two conditions disagree and the desired detectable difference is δ, an independence approximation gives

$$n \approx \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 q}{\delta^2}$$

for a two-sided difference test near the null. For example, **assuming** $q = 0.25$, two-sided $\alpha = 0.05$, power 0.80 and a five-percentage-point difference yields approximately 785 task pairs before accounting for clustering. This is a planning scenario, not an observed discordance rate or a claim that this sample size is sufficient for noninferiority, all policies or all repositories. The final calculation must use the declared test and a conservative dependence assumption.

The approved US$100 GPU allocation is treated as a feasibility resource. A full benchmark across several current models, multiple seeds and closed-provider APIs requires a separately costed acquisition plan. If the funded scope remains a pilot, the final thesis must describe its conclusions as preliminary and narrow its claims accordingly rather than asserting the confirmatory design was executed.

11.6 Prespecified outputs

The completed study will produce five groups of tables and figures:

1. **Cohort and execution accounting:** task/repository counts, exclusions, environment failures, seeds, model/harness revisions and actual resource use.
2. **System outcomes:** resolution, regressions, costs and latency by condition; primary paired contrasts with uncertainty.
3. **Routing diagnostics:** per-model allocation, local/remote exposure, calibration, regret, portfolio-size effects, switching frequency and prespecified subgroups.
4. **Economic scenarios:** API invoice accounting, self-hosted utilization sensitivity, local marginal versus total cost, and adaptation amortization assumptions.
5. **Student and portfolio interventions:** base versus customer-adapted configurations, fixed versus recalibrated router, release-epoch refresh, training/admission cost, export/quantization checks and forgetting diagnostics.

Empty result-table templates are provided in the appendix. No cells are populated from hypothetical values. Published external scores belong in the background model catalog and retain their original attribution; they do not populate the study's result tables.

11.7 Result-dependent discussion to be written later

The final discussion will answer the research questions in the order established here. If quality is retained with lower cost, it will examine the range of workloads and accounting assumptions supporting that result. If cost decreases alongside unacceptable quality loss, it will explain the tradeoff without presenting it as a

successful substitute for the remote model. If differences are unresolved, it will discuss precision and sample size rather than asserting equivalence.

Possible explanations—miscalibration, inadequate features, task shift, weak local tool use, state-transfer overhead, teacher-data bias or serving incompatibility—will require diagnostic evidence. The discussion will distinguish observed mechanisms from plausible hypotheses. It will also assess whether any improvement transfers to unseen repositories, current model revisions and real local hardware. None of those conclusions is written in advance.

11.8 Portfolio and switching diagnostics

The report separates the static single-model frontier, a development-selected domain lookup, the restricted two-model policy, the larger frozen portfolio and the outcome-informed oracle. Pool membership and its data-acquisition cost accompany every curve. A best observed test model is not retrospectively substituted for the development-selected primary comparator. Missing candidate outcomes cannot be filled with invented counterfactual successes.

Switching analysis compares fresh task-level selection with actual phase-level and per-call continuations. It reports cold and warm latency, replayed context tokens, adapter/model load time, switch count, invalidated cache volume where observable, and task failures caused by handoff. API discounts and hardware KV reuse are distinct measurements. A policy saving model tokens while increasing infrastructure time is evaluated under the same declared total-cost accounting.

Customer and release-epoch analyses separate generator adaptation, router recalibration and changes in pool membership. Customer data volume and task volume determine whether training costs can amortize; a cost-only break-even calculation cannot establish that quality is preserved. Later release epochs use new held-out tasks or are labelled exploratory re-evaluations, not fresh confirmations on a repeatedly tuned test set.

Chapter 12

Reproducibility, ethics and limitations

12.1 Implementation state

The research repository contains reusable components, but not all proposed empirical stages have been performed. This distinction matters because a successful import, data preparation or configuration validation is not evidence of language-model quality.

Component	Current status	Evidence it can support
Historical programming-problem data/replay pipeline	Implemented separately	Data validation and retrospective routing methodology on that distribution
Training-example preparation and LoRA scaffold	Implemented for an earlier pinned Qwen configuration	Preparation integrity and software integration; not results for the latest candidate models
Base/adapter generation scaffold	Implemented	A route to fresh inference after model/runtime validation
Tiny random-model training fixture	Software validation only	Loss masking, optimizer and adapter serialization behavior
Current-model catalog and pricing provenance	Source review	Candidate selection and conditional economic assumptions
Common-harness repository-level evaluation	Prospective	No main-study accuracy or cost result yet
Latest-model distillation and RLAIF	Prospective	No empirical improvement claim yet

In particular, an old Transformers or serving-engine pin must not be assumed to support a newly released hybrid-attention or mixture-of-experts architecture. Each selected checkpoint requires a capability preflight: tokenizer/template agreement, generation, stop behavior, output extraction, context limits, model precision and intended adapter targets. A change made to satisfy that preflight is recorded before the relevant run.

Additional portfolio, browser-domain and customer-adapter experiments are designed but have no TTGL performance results. The task-price tables are attributed historical observations or explicitly hypothetical arithmetic, not measurements of these new systems. Supporting document and arithmetic checks validate transcription and calculations only.

12.2 Reproducibility package

The experiment package will include immutable source references, configuration files, data split manifests, environment specifications, policy parameters and execution commands. Each released result is linked to a code revision and dependency lock. Raw artifacts are retained unchanged, and derived artifacts identify their source hashes. A rerun is written to a new directory so it cannot silently overwrite contradictory evidence.

Task selection and treatment assignment are deterministic given the recorded seed and input manifest. Where hosted providers do not guarantee deterministic sampling, repeated outcomes and provider metadata are recorded rather than asserting exact reproducibility. Reproducing a conclusion may be possible even when an identical text trajectory is not. Floating API aliases are a specific threat; the researcher should prefer snapshot IDs and record response-reported model versions when available.

The benchmark evaluator and the agent workspace are separate environments. Container image digests, operating-system packages, test commands, timeouts and resource limits are part of the evidence. Dependency failures are classified according to the prespecified rule. The researcher does not repair only the failed conditions that favor a preferred model; common environment fixes require documented reruns across affected comparisons.

Pricing records include provider, endpoint/model mapping, currency, unit, context tier, cache policy, region, batch status, retrieval date and source. Rates verified only through a community-maintained catalog remain flagged. A price table alone cannot reconstruct an invoice without usage categories. GPU records include creation/deletion timestamps, advertised rate, storage, network charges and actual billing where available.

12.3 Task management and publication

The intended project system is the Fastest Jira board, with manuscript publication in the FASTEST Confluence space. A ticket should reference an experiment stage, its acceptance criteria and immutable artifact locations. State transitions require evidence—for example, a training task is complete only after the declared run completes and its artifacts are verified, not after a script is written.

When the Atlassian connection is unavailable, local task records and import drafts are maintained with clearly marked local identifiers. Synchronization must search for existing work items before creating new ones. Publication is a distinct event from preparing a Markdown or Word document. The manuscript makes no claim that a remote page or issue exists without a confirmed service response.

12.4 Data and research ethics

Public availability is not the same as unrestricted redistribution or training permission. Benchmark tasks, repository code, model weights, software and API-generated outputs can have different licenses. The study records provenance and reviews the applicable terms for each use. A license for an inference kernel does not establish a model-weight license; a model-weight license does not establish a hosted provider's output-training policy.

The initial study uses public benchmark repositories. If later work includes customer code, participation and training permissions must be explicit, with retention, deletion and tenant boundaries defined. Secrets, personal information and proprietary content should not be logged or added to a training set by default. Local-first operation is not a privacy guarantee if the system transmits context to a remote model; remote eligibility remains an enforceable condition of the policy.

Generated code is evaluated in isolated environments with bounded resources. The harness controls tools and credentials independently of model output, and benchmark tests are protected from modification by the agent being assessed. These controls support both reliable measurement and safe execution. They do not establish that a patch passing the benchmark is suitable for production deployment without review.

The research also considers distributional effects of the objective. Maximizing the number of solved issues per dollar may prioritize some languages, repositories or issue types. The study reports coverage and unresolved work across prespecified task strata and does not interpret those strata as a comprehensive social-fairness evaluation. A commercial policy that assigns different value to customers or tasks requires an explicitly different utility function.

12.5 Construct validity

Benchmark resolution is a narrow operational definition of software-engineering accuracy. It measures whether the final patch satisfies specified tests under a particular evaluator. It does not fully measure code readability, maintainability, security, user satisfaction or long-term regressions. A no-tools LLM, a single-model agent and a portfolio-routed agent are different systems; comparisons must name the system-level intervention.

"Confidence" has several meanings: token likelihood, a model's verbal self-assessment, a classifier's predicted success probability and a policy's estimate of utility. Treating them interchangeably weakens the causal argument. The thesis defines the prediction target and evaluates calibration for that target. Speculative-decoding acceptance is not reused as a proof of task correctness.

Cost has similarly different interpretations. An API invoice, marginal electricity, rented GPU time and amortized ownership cost answer different questions. The report states which perspective supports each claim and avoids assigning a zero total cost to user-owned hardware merely because no external API call is billed.

12.6 Internal validity

Major threats include test leakage, inconsistent task environments, different prompt/tool budgets, outcome-dependent exclusions and post-hoc selection of successful models or thresholds. The protocol addresses these through immutable splits, common harnesses, blinded final-test outcomes during development, complete accounting and prespecified contrasts. Known overlaps or near-duplicates are documented rather than erased from the record after their effect becomes visible.

Published paired-output replay can support development, but it does not recreate the effect of an actual stateful escalation. It may also inherit upstream model-training contamination and unknown inference details. The prospective system experiment collects its own complete trajectories and does not infer missing latency or token usage from execution-test metadata.

Distillation adds the risk that the student is evaluated on targets or task families represented in its training data. Verification filtering and teacher-generated preferences create additional selection effects. The study holds final tests aside and compares adaptation controls, while recording which data were available to each component. A frozen router and a recalibrated router are separate conditions.

12.7 External validity

Public benchmarks represent a limited set of repositories, languages and issue types. Repeated use by model developers may make them easier than novel customer work. A temporal task split helps only if the chosen model did not see the relevant tasks during its training or post-training; a recent benchmark date alone does not establish that fact.

The study therefore combines a stable software-engineering benchmark with a prespecified repository-unseen analysis and, if resources permit, external multilingual/frontend evaluations and a separate browser-domain study using their own pinned harnesses. Scores from the external benchmark are reported separately. Failure to reproduce a ranking across benchmarks is a result about transfer, not a reason to silently choose the more favorable benchmark.

Local hardware is another source of variation. GPU memory capacity, memory bandwidth, CPU support, quantization kernels and thermal limits affect latency and practical model choice. A single 80 GB cloud GPU is useful for training and controlled inference but is not evidence about ordinary laptops. The final product claim requires separate device measurements or a clearly restricted hardware scope.

Portfolio breadth creates additional external-validity limits. A router fitted to a few eligible models is not evidence about every free or open-weight release. A dataset dominated by one language cannot establish language specialization, and browser navigation scores cannot establish frontend implementation quality. A customer adapter evaluated on its own training templates cannot establish transfer to new customer workflows. New release epochs and new customer cohorts require fresh development and held-out evidence.

12.8 Resource and completion boundaries

The user has authorized a Runpod experiment ceiling of US$100 for one 80 GB GPU. The pilot must reserve resources for startup, model download, artifact export and verified cleanup. A timeout on a training process does not stop GPU billing. Provisioned resources are tracked and terminated through the provider, with removal confirmed. Additional provider APIs and a full multi-model study require their own resource plan; they are not silently treated as covered by that GPU budget.

The main study is complete only when its declared conditions have been executed, failures accounted for, invoices and usage reconciled, statistical analysis performed and results written without replacing the prespecified questions. If access or funding supports only a pilot, the thesis must identify that scope explicitly. Unexecuted configurations, synthetic examples and software tests cannot be promoted into empirical model-performance evidence.

Results and discussion reserved for completion

No empirical results chapter is included in this draft. After execution, it will report the complete cohort, all prespecified conditions, primary contrasts, uncertainty, calibration, cost accounting, student adaptation and failures. The discussion will interpret those observations against the research questions and limitations. The conclusion will be written only after the evidence exists.

Appendix: empty reporting templates

Main system comparison

Condition	Tasks attempted	Resolution	Total cost per issue	Cost per resolved issue	Uncertainty
Local-only agent	Pending	Pending	Pending	Pending	Pending
Open-weight remote-only agent	Pending	Pending	Pending	Pending	Pending
Closed-model remote-only agent	Pending	Pending	Pending	Pending	Pending
Restricted two-model policy	Pending	Pending	Pending	Pending	Pending
Frozen multi-model portfolio	Pending	Pending	Pending	Pending	Pending
Development-selected domain lookup	Pending	Pending	Pending	Pending	Pending
Random policy matched on calibration cost	Pending	Pending	Pending	Pending	Pending

Produce this table separately for each benchmark/domain, deployment scenario and portfolio epoch. The fixed primary reference is chosen on development data. Add explicit columns or linked manifests for adapter identity, cold/warm state and cost boundary; no historical accuracy score is inserted into an unrun condition.

Routing and student diagnostics

Quantity	Prespecified interpretation	Observation
Local coverage	Tasks completed without remote model assistance	Pending
Calibration	Predicted versus observed task/continuation success	Pending
Routing gain	Paired utility difference at comparable cost	Pending
Specialist contribution	Remove-one-model ablation and added maintenance cost	Pending

Quantity	Prespecified interpretation	Observation
Switching effect	Task-level versus actual phase/call handoff	Pending
Portfolio refresh	Frozen epoch versus development-selected new epoch	Pending
Student effect	Base versus adapted student under the same harness	Pending
Router adaptation effect	Frozen versus recalibrated router after student update	Pending
Amortization	Training cost relative to measured eligible savings	Pending

Run manifest checklist

Required fields are task, repository and base commit; benchmark revision; harness commit; image digest; condition; portfolio epoch and eligibility set; model, adapter and tokenizer revisions; decoding configuration; seed where supported; routing features and decision; context, tool, iteration and time limits; usage categories; actual billed rates; final patch; evaluator output; failure class; start and end timestamps; resource cleanup; artifact hashes; and analysis-code revision. Unknown values remain explicitly unknown.

References

AgentLab contributors. 2026. ‘AgentLab: Web-Agent Development and Benchmarking’. Versioned repository documentation. https://github.com/ServiceNow/AgentLab/blob/8488c564eea0525aaff5dea189bb03a9b529546e/README.md.

Bai, Yuntao et al. 2022. ‘Constitutional AI: Harmlessness from AI Feedback’. https://arxiv.org/abs/2212.08073.

Browser Use contributors. 2026a. ‘Browser Use Bench V2.1: 200-Task Benchmark and Historical Comparison Caveats’. Versioned repository documentation. https://github.com/browser-use/benchmark/blob/77b4df339fe8fc293fb48896cfa1c73dd743af42/README.md.

———. 2026b. ‘Browser Use: Local and Hosted Browser-Agent Library Documentation’. Versioned repository documentation. https://github.com/browser-use/browser-use/blob/3df5806a0d097a1546d92c49bcf1b28c9972c3bd/README.md.

BrowserGym contributors. 2026. ‘BrowserGym: Browser Environments for Web-Agent Research’. Versioned repository documentation. https://github.com/ServiceNow/BrowserGym/blob/4ccc72332ab9882bc9aed87264dbf60b16e426f0/README.md.

ByteDance. 2026. ‘UI-TARS: Released GUI-Agent Checkpoints and Model-Scale Comparisons’. Versioned repository documentation. https://github.com/bytedance/UI-TARS/blob/582f3a7ea5d285ee8ed9e2e84048d1ab01453c49/README.md.

Chen, Lequn, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2023. ‘Punica: Multi-Tenant LoRA Serving’. https://arxiv.org/abs/2310.18547.

Chen, Lingjiao, Matei Zaharia, and James Zou. 2024. ‘FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance’. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=cSimKw5p6R.

Chen, Mark et al. 2021. ‘Evaluating Large Language Models Trained on Code’. https://arxiv.org/abs/2107.03374.

DeepSeek-AI. 2025. ‘DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning’. https://github.com/deepseek-ai/DeepSeek-R1/blob/main/DeepSeek_R1.pdf.

———. 2026. ‘DeepInfer: High-Performance, Large-Scale Inference Framework for DeepSeek Models’. Source code and documentation. https://github.com/deepseek-ai/DeepInfer/tree/553741557a57df44a46fb3ab72b6aafeaf945337.

Deng, Xiang, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, et al. 2025. ‘SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?’ *arXiv Preprint arXiv:2509.16941*. https://arxiv.org/abs/2509.16941.

Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. 'QLoRA: Efficient Finetuning of Quantized LLMs'. *arXiv Preprint arXiv:2305.14314*. https://arxiv.org/abs/2305.14314.

Ding, Dujian, Ankur Mallick, Shaokun Zhang, Chi Wang, Daniel Madrigal, Mirian Del Carmen Hipolito Garcia, Menglin Xia, Laks V. S. Lakshmanan, Qingyun Wu, and Victor Rühle. 2025. 'BEST-Route: Adaptive LLM Routing with Test-Time Optimal Compute'. *arXiv Preprint arXiv:2506.22716*. https://arxiv.org/abs/2506.22716v1.

Feng, Tao, Yanzhen Shen, and Jiaxuan You. 2025. 'GraphRouter: A Graph-Based Router for LLM Selections'. In *International Conference on Learning Representations*. https://arxiv.org/abs/2410.03834v2.

Feng, Tao, Fangxu Yu, Haozhen Zhang, Zhongjie Dai, Liangqi Yuan, Zijie Lei, Weizhi Zhang, et al. 2026. 'LLMRouter: Unified Infrastructure for Developing, Evaluating, and Deploying LLM Routers'. *arXiv Preprint arXiv:2608.06867*. https://arxiv.org/abs/2608.06867v1.

Geifman, Yonatan, and Ran El-Yaniv. 2019. 'SelectiveNet: A Deep Neural Network with an Integrated Reject Option'. In *Proceedings of the 36th International Conference on Machine Learning*. https://proceedings.mlr.press/v97/geifman19a.html.

Greenewald, Kristjan, Luis Lastras, Thomas Parnell, Vraj Shah, Lucian Popa, Giulio Zizzo, Chulaka Gunasekara, Ambrish Rawat, and David Cox. 2025. 'Activated LoRA: Fine-Tuned LLMs for Intrinsics'. https://arxiv.org/html/2504.12397v3.

Guo, Chuan, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. 'On Calibration of Modern Neural Networks'. In *Proceedings of the 34th International Conference on Machine Learning*. https://arxiv.org/abs/1706.04599.

Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean. 2015. 'Distilling the Knowledge in a Neural Network'. https://arxiv.org/abs/1503.02531.

Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. 'LoRA: Low-Rank Adaptation of Large Language Models'. https://arxiv.org/abs/2106.09685.

Hu, Qitian Jason, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. 2024. 'RouterBench: A Benchmark for Multi-LLM Routing System'. *arXiv Preprint arXiv:2403.12031*. https://arxiv.org/abs/2403.12031v2.

Jain, Naman, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. 'LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code'. https://github.com/LiveCodeBench/LiveCodeBench.

Jimenez, Carlos E., John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. 'SWE-Bench: Can Language Models Resolve Real-World GitHub Issues?' In *International Conference on Learning Representations*. https://openreview.net/forum?id=VTF8yNQM66.

Leviathan, Yaniv, Matan Kalman, and Yossi Matias. 2023. 'Fast Inference from Transformers via Speculative Decoding'. In *Proceedings of the 40th International Conference on Machine Learning*, 202:19274–86. https://proceedings.mlr.press/v202/leviathan23a.html.

Liu, Jiawei, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. 'Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation'. In *Advances in Neural Information Processing Systems*. https://openreview.net/forum?id=1qvx610Cu7.

Mamou, Jonathan, Oren Pereg, Daniel Korat, Moshe Berchansky, Nadav Timor, Moshe Wasserblat, and Roy Schwartz. 2024. 'Dynamic Speculation Lookahead Accelerates Speculative Decoding of Large Language Models'. *arXiv Preprint arXiv:2405.04304*. https://arxiv.org/abs/2405.04304.

Microsoft. 2026. 'Playwright: Cross-Browser Testing and Automation'. Versioned repository documentation. https://github.com/microsoft/playwright/blob/a2b65a057a78204d8411b068e1e28d0db23db44b/README.md.

Mozannar, Hussein, and David Sontag. 2020. 'Consistent Estimators for Learning to Defer to an Expert'. In *Proceedings of the 37th International Conference on Machine Learning*. https://arxiv.org/abs/2006.01862.

Ong, Isaac, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2025. 'RouteLLM: Learning to Route LLMs with Preference Data'. In *International Conference on Learning Representations*. https://arxiv.org/abs/2406.18665.

OSWorld contributors. 2026. 'OSWorld and OSWorld-Verified: Computer-Use Benchmark Documentation'. Versioned repository documentation. https://github.com/xlang-ai/OSWorld/blob/044bc33901cb893808e3f91ebc9a17ed8e54b44c/README.md.

Rafailov, Rafael, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. 'Direct Preference Optimization: Your Language Model Is Secretly a Reward Model'. In *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2305.18290.

Scale AI. 2026. 'SWE-Bench Pro V2: Validated Public Split and Locked Evaluation Protocol'. Versioned benchmark documentation. https://github.com/scaleapi/SWE-bench_Pro-os/blob/66f92766bba642462d4bbe5479e83f91f9211862/v2/README.md.

Shao, Zhihong, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. 'DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models'. *arXiv Preprint arXiv:2402.03300*. https://arxiv.org/abs/2402.03300.

Sheng, Ying, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, et al. 2023. 'S-LoRA: Serving Thousands of Concurrent LoRA Adapters'. https://arxiv.org/html/2311.03285v3.

Stanford SALT lab. 2026. 'Design2Code: Benchmark and Implementation Documentation'. Versioned official repository documentation. https://github.com/NoviScl/Design2Code/blob/7a575e4c33f417c4be5c64072b8f5798de0d0f99/README.md.

SWE-agent Team. 2026. 'Mini-SWE-Agent: The Minimal AI Software Engineering Agent, Version 2 Documentation'. Versioned software documentation. https://github.com/SWE-agent/mini-swe-agent/blob/04d809ceab9df28f9adaed044884180159172930/README.md.

SWE-bench contributors. 2026. 'SWE-Bench Multilingual: Dataset Card and Evaluation Schema'. Versioned dataset documentation. https://huggingface.co/datasets/SWE-bench/SWE-bench_Multilingual/blob/846e647b9f33c0b51b739d005d13d85493c9af09/README.md.

SWE-bench Team. 2024. 'SWE-Bench Verified and Evaluation Documentation'. Official project documentation. https://github.com/SWE-bench/SWE-bench/blob/02e7a74ffd0b707aab73d203fe87bdc7c76afc8e/README.md.

VisualWebArena contributors. 2026. 'VisualWebArena: Multimodal Web-Agent Benchmark Documentation'. Versioned repository documentation. https://github.com/web-arena-x/visualwebarena/blob/89f5af29305c3d1e9f97ce442146

2060a70c9a03/README.md.

Wang, Xingyao, Simon Rosenberg, Juan Michelini, Calvin Smith, Hoang Tran, Engel Nyst, Rohit Malhotra, et al. 2025. 'The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents'. https://arxiv.org/abs/2511.03690.

WebArena contributors. 2026. 'WebArena: Canonical Benchmark Repository and Implementation Guidance'. Versioned repository documentation. https://github.com/web-arena-x/webarena/blob/dce04686a56253aefba7b18a4fa0937cf1dc987b/README.md.

WebArena-Verified contributors. 2026. 'WebArena-Verified: Versioned Web Tasks and Deterministic Trace-Based Evaluation'. Versioned repository documentation. https://github.com/ServiceNow/webarena-verified/blob/6473f72db5dcefc97b5725b59e734504edc28a21/README.md.

WorkArena contributors. 2026. 'WorkArena and WorkArena++: Enterprise Browser-Task Benchmark Documentation'. Versioned repository documentation. https://github.com/ServiceNow/WorkArena/blob/e885130962f61fb64ac305497996a46cfc7d18f0/README.md.

Yang, John, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. 2024. 'SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering'. In *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2405.15793.

Zhuo, Terry Yue, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, et al. 2024. 'BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions'. *arXiv Preprint arXiv:2406.15877*. https://arxiv.org/abs/2406.15877.